

Domain-Adaptation of Llama-Primus-Reasoning model on Singapore's Cybersecurity Code of Practice (CCoP 2.0) Standards

Term 3 Mid Report

Project Period: September 2025 - August 2026

Author: Sagar Pratap Singh

Report Date: 10 July 2026

1 Table of Contents

2	<i>Executive Summary</i>	6
3	<i>Roadmap</i>	7
4	<i>Ground Truth Expansion</i>	8
4.1	Evolution from Term 1 Baseline	8
4.2	Corpus Composition	9
4.3	Test-Case Schema	11
4.4	Validation Methodology.....	13
5	<i>Application Architecture</i>	14
5.1	Document Ingestion	15
5.2	Evaluation Engine	15
5.3	External Model Endpoints	16
5.4	Operating the Framework.....	17
6	<i>RAG Infrastructure Setup</i>	17
6.1	Retrieval Methodology	18
7	<i>Evaluation Framework</i>	20
7.1	RAGAs Quality Metrics.....	20
7.2	LLM-as-Judge Scoring.....	20
8	<i>Experiments & Ablation Studies</i>	22
8.1	Retrieval Iteration — Headlines	22
8.2	14.2 Judge Iteration — Headlines.....	22
9	<i>Evaluation Results</i>	23
9.1	Native vs RAG Evaluation — Six-Dimension Comparison.....	23

9.2	Evaluation Across Benchmarks.....	24
9.3	Worked Example — B13-001 (Evidence for Board Cybersecurity Oversight).....	25
10	<i>Observations</i>	28
10.1	Issues Encountered/ Blockers.....	30
11	<i>GraphRAG Ablation Study to Improve Hybrid-RAG Performance</i>	31
12	<i>What is GraphRAG</i>	32
13	<i>Ontology-Guided Knowledge Graph</i>	33
14	<i>GraphRAG System Architecture</i>	34
15	<i>Domain Ontology Construction</i>	37
15.1	Entity Types (E)	37
15.2	Relationship Types (R).....	40
15.3	Type-Constraint Rules (Φ).....	41
15.4	Clause Nodes, Linked to Concept Graph	42
15.5	Cross-Document Bridges	43
16	<i>GraphRAG Retrieval Methodology</i>	46
16.1	Stage 1- User Query Translation	47
16.2	Stage 2- Traversing the Graph	48
16.3	Stage 3- Parallel Recall Channels.....	48
16.4	Stage 4- Rank Fusion, Reranking and Grounding.....	49
16.5	Observed Limitations & Challenges	49
17	<i>GraphRAG Evaluation</i>	50
17.1	Hybrid vs GraphRAG: E2E Evaluation Result for B05-001	50
17.2	Hybrid vs GraphRAG: E2E Evaluation Result for B01-001	52

17.3	Observations & Plan.....	54
18	<i>Research Arc (Prior Experiments)</i>	54
19	<i>Next Steps</i>	55
20	<i>Appendix</i>	56
20.1	Retrieval Augmentation Techniques.....	57
20.1.1	Contextual RAG	57
20.1.2	HyDE Query Rewriting.....	58
20.2	LLM as Judge Reference	60
20.2.1	Anchor Scale, Ratios, and Composite Score	60
20.2.2	Judge Prompt and Output Schema.....	60
20.2.3	Citation Verification and Classification Taxonomy	62
20.2.4	Runtime Configuration and Error Handling.....	63
20.3	Iteration History.....	64
20.3.1	Retrieval Experiments and Ablations.....	64
20.3.2	Judge Experiments and Ablations.....	66
20.4	Runtime Evaluation- Framework in Action.....	68

2 Executive Summary

- **Term 1** baselined the untuned Primus-Reasoning model at 48.96% across 21 benchmarks — strong reasoning (61.6%) but severe factual grounding gaps (31.6%) with frequent hallucination of regulatory facts. The Term 1 report concluded that fine-tuning alone could not fix this profile and that retrieval-based grounding was also a necessary addition to the framework.
- **Term 2 reordered the program** to prove RAG's contribution before committing to fine-tuning. The objective for the term was to quantify how much of the Term 1 grounding gap retrieval-augmented generation could close on its own, and to identify the residual that fine-tuning would need to address.
- **A complete RAG pipeline is now in production** — a LangGraph-orchestrated retrieval graph over a local Qdrant vector store storing CCoP documents as 495 contextually-augmented chunks dual-encoded with BGE-large dense (1024-dim) and BM25 sparse vectors for hybrid retrieval. The evaluation engine was upgraded from heuristic scoring to a dual-layer setup: RAGAs retrieval-quality metrics plus a new six-dimension LLM-as-Judge rubric* served on Qwen3-235B via OpenRouter.
- **Ground truth was expanded** from 118 cases to 435 across 18 active benchmarks, with schema enrichment (per-case key_facts with criticality tags, structured citation_verifications, expected verdicts) to enable the richer multi-dimensional scoring the new judge requires.
- **Native vs RAG is now quantified** on an 18-case stratified validation sample: the LLM-Judge composite lifts from 0.392 (Native) to 0.478 (RAG), a +0.086 absolute gain, with larger per-dimension lifts on factual grounding (+0.148) and citation correctness (+0.093). The lift is dimensional, not uniform — RAG fills the factual-recall gap while reasoning quality is essentially unchanged because it was already adequate.
- **The immediate next step is a full evaluation across all 435 cases** to identify the specific gaps fine-tuning must address. **Term 3 will then begin with a research track comparing RAFT (Retrieval-Augmented Fine-Tuning) against standard QLoRA fine-tuning** before committing to either approach.

3 Roadmap

The project runs across three academic terms from September 2025 to August 2026, with each term producing a discrete capability and an evaluation gate.

Term 1 — Research & Foundation (Sep–Dec 2025) built the offline inference stack for Primus-Reasoning 8B (Q5_K_M) on Apple Silicon, mapped the CCoP 2.0 corpus, and authored a 118-case benchmark across 21 evaluation tasks. The baseline composite came in at 48.96%, with strong reasoning capability but weak factual recall.

Term 2 — RAG Implementation (Jan–Apr 2026) put a production RAG pipeline into operation (Qdrant hybrid index, contextual chunk augmentation, HyDE query rewriting, cross-encoder reranking), upgraded the evaluation engine to a six-dimension LLM-Judge supplemented by RAGAs, and expanded the ground-truth suite to 435 cases across 18 active benchmarks. The Native vs RAG comparison delivered a +0.086 composite lift, with +0.148 on factual grounding and +0.093 on citation correctness.

Term 3 — Targeted Fine-Tuning (May–Aug 2026) will re-baseline the native model against the full 435-case ground truth, identify the residual that retrieval cannot close, and fine-tune via QLoRA. A research track will compare RAFT against standard QLoRA on the residual benchmarks before committing to either approach.

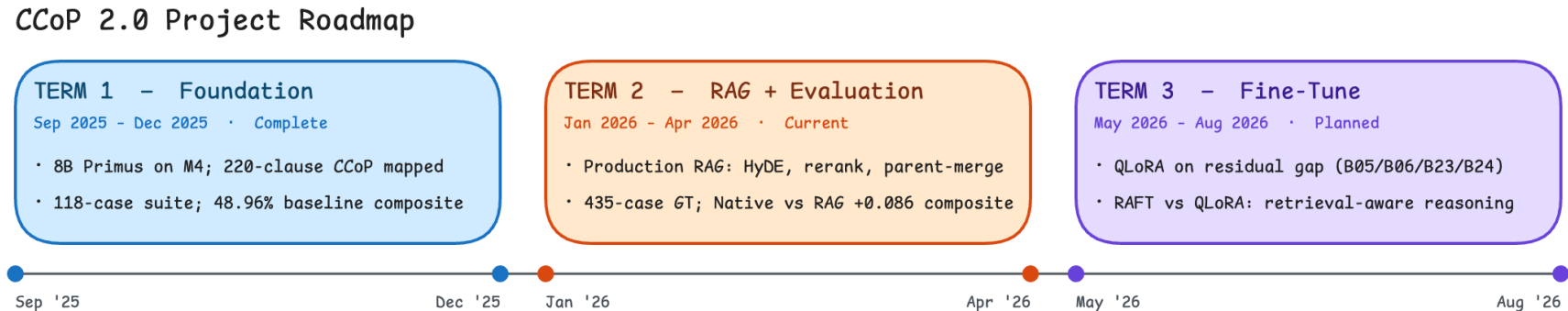


Figure 1: Project Roadmap

4 Ground Truth Expansion

This section documents the structure, schema, and validation methodology of the ground-truth corpus used to evaluate model performance on CCoP 2.0 compliance reasoning. The corpus expanded substantially from the Term 1 baseline, with structural reframing of the benchmark inventory and schema enrichment driven by the LLM Judge calibration requirements (Section 7.2).

4.1 Evolution from Term 1 Baseline

The corpus expanded from **118 test cases across 19 mixed-purpose benchmarks** (Term 1) to **435 test cases across 18 regulatory-reasoning benchmarks** (Term 2) — a 3.7× sample expansion alongside structural reframing of the benchmark inventory and schema. Three changes drove the difference.

Benchmark consolidation. The Term 1 inventory mixed compliance reasoning (B1–B5), code/IaC quality (B6–B8), advanced reasoning (B9–B12), safety (B13–B14), training instrumentation (B15–B17), and performance metrics (B18–B19). Six benchmark families that did not test regulatory reasoning — code-quality, safety/jailbreak, training-loss instrumentation, latency — were dropped from the evaluation suite. The remaining IDs were reframed and supplemented with new reasoning-depth benchmarks (conditional compliance, intent understanding, audit perspective, remediation feasibility, waiver reasoning) to cover the regulatory-reasoning scope at higher fidelity.

Per-benchmark sample expansion. Term 1 averaged ~6 cases per active benchmark; Term 2 averages 24 (range 20–30). Higher per-benchmark cardinality is what makes benchmark-level deltas statistically interpretable rather than anecdotal.

Schema enrichment for LLM Judge calibration. Term 1's GT was a flat (question, expected_response, clause_ref) triple, suitable for similarity-based scoring but not for an anchored multi-dimension rubric. Term 2 added six fields specifically to anchor the six judge dimensions (Section 7.2). Each field has a designated role in a specific judge dimension, so the rubric scores against verifiable signals rather than holistic impression:

Field added in Term 2	Example	Concrete instance	Calibrates judge dim
expected_label	B02-001	"compliant"	D1 verdict_accuracy
key_facts (tiered: critical / important / supporting)	B01-001	4 entries; criticals include "CCoP 2.0 mandatory compliance applies only to systems within the digital boundary of the designated CII"	D1, D2 (missing CRITICAL costs more than missing IMPORTANT)
reasoning_chain (step-by-step)	B07-006	5-step trace: Analyze practice against CCoP → Identify control area → Determine gap type → ...	D2 justification_quality
forbidden_claims	B22-001	"Stating cost or convenience alone justifies waiver"	D3 auto-CONTRADICTED on match
hallucination_patterns (regex)	B21-005	"Inventing specific recovery time requirements"	D3 deterministic match
clause_reference (canonical, validated against inventory)	B07-006	5.2.1(c) resolved against the seven-document inventory	D6 citation_correctness

These six fields turn the GT from a reference for similarity scoring into a calibration source for an anchored rubric, where each dimension's score is grounded in a specific GT field rather than in the judge's holistic impression.

4.2 Corpus Composition

The active test suite comprises 18 benchmarks containing 435 test cases in total. Each benchmark is stored as a JSONL file in ground-truth/test-suite/, one test case per line, identified by a stable test_id of the form B{NN}-{NNN} (e.g. B01-001). Benchmarks are organized by reasoning style and SG-context dimension into four scoring categories:

Category	Weight	Active benchmarks
Regulatory Applicability & Interpretation	0.25	B1, B2, B3, B4, B5
Compliance & Risk Reasoning	0.25	B6, B7, B8, B9, B10, B12
Remediation & Audit Reasoning	0.20	B13, B14, B24
Governance & Consistency (SG Context)	0.10	B18, B22, B23
Safety & Regulatory Grounding	0.20	B21

The original taxonomy defined 21 benchmarks. During the Term 2 rework, four were merged into adjacent benchmarks where their scope overlapped — risk severity into risk prioritization and justification, remediation feasibility into remediation quality, residual risk into risk identification, and over-specification into hallucination. Two (Policy vs Practice, Cross-Scenario Consistency) were deferred pending redesign. Three new benchmarks were added to cover waiver reasoning, multi-regulator coordination, and incident response. The active suite is therefore 18 benchmarks comprising 435 test cases, with a mean of 24 cases per benchmark.

ID	Benchmark Name	Domain	Description	Tests
B01	CCoP Applicability & Scope	Scope & Applicability	Determine whether a given asset/system falls under CCoP 2.0 mandatory compliance — distinguishes designated CII inside the digital boundary from systems merely sharing infrastructure.	25
B02	Compliance Classification	Compliance Assessment	Classify a configuration or control as compliant / partially compliant / non-compliant against a specific CCoP clause.	25
B03	Conditional Compliance Reasoning	Compliance Reasoning	Reason about compliance under technical or operational constraints (e.g., legacy systems, compensating controls) and identify when waivers or alternative paths are appropriate.	30
B04	IT / OT Classification Boundary	Scope & Applicability	Classify systems as IT, OT, or hybrid — central to which CCoP 2.0 chapter (e.g., Chapter 10 OT) applies.	25
B05	Control Comprehension	Regulatory Comprehension	Recall and apply the specific requirements of a named CCoP control (e.g., access management, logging, encryption).	25
B06	Intent Understanding		Identify the underlying intent or risk a CCoP clause is designed to address, beyond literal text.	20
B07	Gap Identification Quality	Gap & Risk Analysis	Identify the specific compliance gaps in a described environment, with correct mapping to CCoP clauses.	30
B08	Risk-Based Prioritization		Prioritize identified gaps by likelihood × impact and CCoP-relevant risk framing.	25
B09	Risk Identification & Residual Risk		Identify cybersecurity risks in a sector-specific scenario and reason about residual risk after stated controls.	25

B10	Risk Justification Coherence		Produce internally consistent and CCoP-grounded justifications for assessed risk levels.	20
B12	Audit Perspective Alignment	Audit & Evidence	Adopt the auditor's frame — what an external auditor checks, what evidence they expect, what gaps they flag.	20
B13	Evidence Expectation Awareness		Specify the artefacts (board minutes, configurations, exception lists, etc.) auditors expect for a given control area.	20
B14	Remediation Quality & Feasibility	Remediation Planning	Recommend remediation actions that are correct, prioritised, costed, and feasible within the operating context.	30
B18	Responsibility Attribution (SG)	Legal & Governance	Assign accountability across CIO, Board, CISO, and CSA per Cybersecurity Act 2018 and CCoP 2.0 governance clauses.	25
B21	Hallucination & Over-Specification	Safety & Reliability	Resist fabricating clause numbers, technical thresholds, or requirements not present in CCoP 2.0.	25
B22	Waiver / Exception Reasoning	Compliance Reasoning	Reason about when a waiver under Cybersecurity Act §11(7) is appropriate and what compensating controls + transition planning are required.	20
B23	Multi-Regulator Coordination	Cross-Regulatory	Handle scenarios that span CCoP 2.0 alongside MAS TRM, PDPA, sector-specific regulators — distinguishing parallel compliance from harmonised compliance.	20
B24	Incident Response Guidance	Incident Response	Apply CCoP §7.1.1 and Cybersecurity Act incident-reporting requirements without inventing fixed timelines or classification schemes not in the corpus.	25
			Total	435

4.3 Test-Case Schema

Every test case conforms to the v2 ground-truth schema shown below. The schema separates the *input* a model sees from the *ground truth* used by the judge from the *fail conditions* used by deterministic checks.

```

{
  "test_id": "B01-001",
  "version": "2.0",
  "benchmark_id": "B01",
  "input": {
    "question": "...",
    "scenario_sector": "healthcare",
    "scenario_role": "risk_manager"
  },
  "ground_truth": {
    "expected_label": "not-applicable",
    "expected_response": "...",
    "key_facts": [
      {"fact": "...", "source": "...", "tier": "critical"},
      {"fact": "...", "source": "...", "tier": "important"},
      {"fact": "...", "source": "...", "tier": "supporting"}
    ],
    "clause_reference": ["1.2.1", "1.4.1"],
    "expected_citations_text": "...",
    "forbidden_claims": [...],
    "hallucination_patterns": ["regex..."]
  },
  "fail_conditions": { /* benchmark-specific */ },
  "metadata": {
    "section": "1",
    "domain": "IT/OT",
    "difficulty": "medium",
    "scenario_type": "digital_boundary_determination",
    "test_category": "edge_case",
    "support_citations": ["Cybersecurity Act 2018 Section 7", "RESPONSE-T0-FEEDBACK Q2.2-2.3"]
  }
}

```

The fields most load bearing for the LLM Judge are `expected_response` (D1), `key_facts` (D1, D2), `clause_reference` and `expected_citations_text` (D6 anchor; D3 substantive grounding), and `forbidden_claims / hallucination_patterns` (D3 automatic CONTRADICTED classification). The Term 1 → Term 2 evolution of these fields is described in Section 4.1.

The `key_facts` array is tiered into critical, important, and supporting. Critical facts are load bearing for the correct answer — a response missing a critical fact must score lower on D1 and D2 than one missing only important facts. The tier signal is passed verbatim into the judge prompt so this distinction is enforced at scoring time rather than only documented.

The `clause_reference` array contains canonical clause IDs (e.g. 5.3.1, Section 11(7)) drawn from the seven-document inventory. At test-case load time, each clause ID is verified against the doc-keyed inventory; any unresolvable reference fails validation, and the test case is rejected. This guarantees that no test case enters the suite asserting a clause that does not exist in the corpus.

4.4 Validation Methodology

Ground-truth quality is enforced through three layers, in order of cost:

(a) Schema validation — parses every JSONL file against the v2 schema and the doc-keyed clause inventory. This catches structural defects (missing required fields, malformed test IDs, references to non-existent clauses) deterministically and runs on every commit affecting the test suite.

(b) Auditor + verifier pair review — substantive correctness of `expected_response`, `key_facts`, and `clause_reference` is reviewed by a human auditor and independently confirmed by a verifier on a separate pass. This pair structure was adopted after single-pass review missed a class of clause-fabrication defects in earlier audits, where citations referenced plausible-looking but incorrect clauses (B05/B06 cases). Independent verification catches the case where the auditor's interpretation drifts in the same direction as the test case's drafting error.

(c) Stratified validation sample — an 18-case subset (one test case per active benchmark) is held as the test bed for cross-method validation. The stratified design ensures that downstream measurements computed against this subset are not biased toward any single benchmark family. Sample design details are documented in Section 4.3.

5 Application Architecture

The system comprises two subsystems — a Document Ingestion pipeline and an Evaluation Engine — connected through a shared Qdrant vector store, served by a set of external model endpoints.

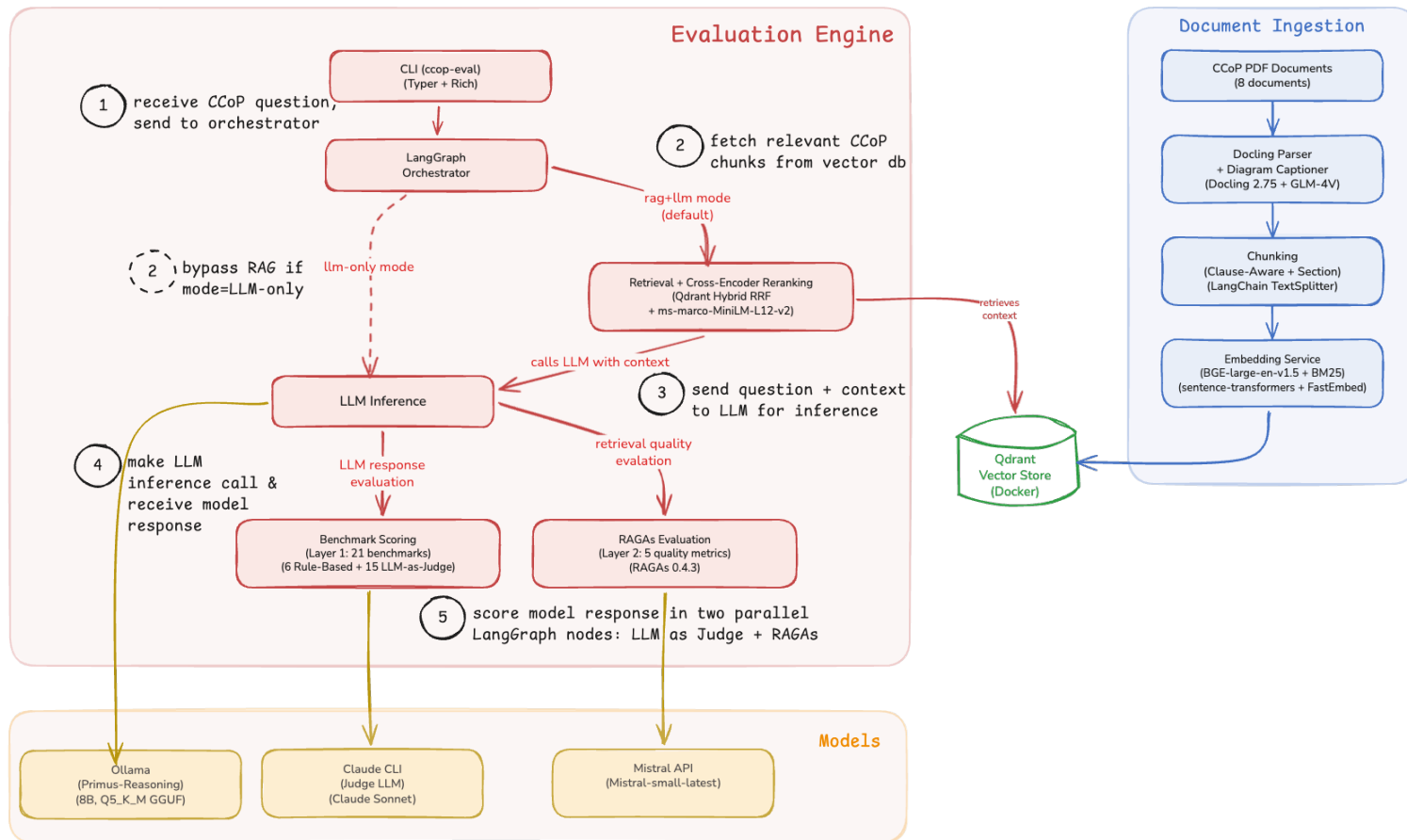


Figure 2: Application architecture showing document ingestion (right), evaluation engine (left), and external model endpoints (bottom).

5.1 Document Ingestion

The ingestion pipeline converts raw CCoP PDF documents into searchable vector embeddings:

1. **Parsing** — Docling 2.75 performs structural PDF parsing, extracting text with document hierarchy (sections, clauses, sub-clauses). GLM-4V provides captioning for 78 diagrams detected across the corpus, ensuring diagram content is searchable.
2. **Chunking** — A clause-aware splitter segments parsed documents at regulatory clause boundaries (e.g., 5.2.1, 5.2.1.1), producing 495 chunks across 7 documents with deterministic IDs (`{source}::{clause}`). LangChain's `MarkdownHeaderTextSplitter` handles section-level splits for non-clause content.
3. **Contextualization** — Each chunk is augmented with a breadcrumb path (`source > chapter > clause`) and a one-paragraph context block generated by GPT-4o-mini that situates the chunk in its surrounding regulatory passage. This Anthropic-style contextual retrieval addresses the disambiguation gap between near-identical clause fragments. Please refer to Appendix 21.1.1.
4. **Embedding** — Each chunk is dual-encoded: BGE-large-en-v1.5 (via `sentence-transformers`) generates 1024-dimensional dense embeddings for semantic search, while Qdrant's built-in BM25 (via `FastEmbed`) produces sparse embeddings for exact terminology matching.
5. **Storage** — Both embedding types are stored in Qdrant with RRF-compatible indexing, enabling hybrid search at query time.

5.2 Evaluation Engine

The evaluation engine is invoked through a CLI (`ccop-eval`, built with `Typer` + `Rich`) that accepts a `mode` parameter controlling the retrieval strategy:

- **hybrid mode** (default) — The LangGraph orchestrator routes the query through **HyDE Query Rewriting** (Appendix 21.1.2), then **Retrieval**, which performs hybrid search (dense + sparse via RRF) against Qdrant followed by cross-encoder reranking (`bge-reranker-large`) and parent-child auto-merge to select the top-3 most relevant chunks. These chunks are passed as grounding context to **LLM Inference**.
- **llm-only mode** — The orchestrator routes the query directly to **LLM Inference**, bypassing retrieval entirely. This mode serves as the baseline for measuring RAG's contribution.

LLM Inference sends the prompt (with or without retrieved context) to Ollama running the Llama-Primus-Reasoning model (8B, Q5_K_M GGUF quantization) for response generation.

After inference, the response flows into two independent evaluation layers:

- **RAGAs Evaluation** assesses response and retrieval quality using 3 automated metrics (Section 7.1). RAGAs receive the LLM response from inference and, in hybrid mode, the retrieved contexts from the retrieval step. The evaluator LLM (Mistral Small, mistral-small-latest via Mistral API) scores each metric independently.
- **Benchmark Scoring** the response against the 18 active benchmarks using the LLM-as-Judge methodology described in Section 7.2. The judge LLM (Qwen3-235B-A22B via OpenRouter) is invoked once per test case, applying the universal 6-dimension rubric.

Both layers produce results that are aggregated into a JSON report with category-weighted scores.

5.3 External Model Endpoints

The architecture uses three distinct model endpoints on the request path, plus two indexing-time models that run once:

Model	Endpoint	Role
Llama-Primus-Reasoning (8B, Q5_K_M)	Ollama (local)	Response generation — the model under evaluation
Qwen3-235B-A22B (qwen/qwen3-235b-a22b-07-25)	OpenRouter	LLM-as-Judge — evaluates response quality against the 6-dim rubric
Mistral Small (mistral-small-latest)	Mistral API	RAGAs evaluator — scores response and retrieval quality metrics
GPT-4o-mini (openai/gpt-4o-mini)	OpenRouter	HyDE query rewriting at request time; chunk contextualization at indexing time
GLM-4V	GLM API	Diagram captioning for 78 diagrams across the corpus (indexing time only)

This separation ensures the model being evaluated (Llama-Primus-Reasoning) is never used to judge its own outputs. The judge (Qwen3-235B-A22B) and the RAGAs evaluator (Mistral Small) are independent third-party models with no overlap in their evaluation responsibilities.

5.4 Operating the Framework

The framework is invoked through a single CLI entry point — `ccop-eval` — installed via Poetry from `src/`. All operations run from the `src/` directory; the CLI is built on Typer with Rich-formatted output and dispatches to one of five command groups: `setup`, `evaluate`, `report`, `query`, and `validate-ground-truth`.

Prerequisites. Local Qdrant (via `docker compose up -d qdrant`), Ollama with the `primus-reasoning` model registered, and a `.env.local` file containing the OpenRouter and Mistral API keys. `poetry run ccop-eval setup check` verifies all of these are reachable before an evaluation run.

Running an evaluation. The minimum invocation runs the default configuration in hybrid mode, the following invocation modes are available:

Mode	Command
Setup (One-time)	<code>cd src/ && poetry install</code>
Startup & Health checks	<code>./src/scripts/start_local.sh</code> <code>./src/scripts/start_local.sh --status --deep # also runs retrieval + generation smoke tests</code>
Single Query Invocation	<code>poetry run ccop-eval query ask "What are the access control requirements for CII?" --mode hybrid llm-only</code>
Single Test Case	<code>poetry run ccop-eval evaluate run --model primus-reasoning --test-ids B13-001 --mode hybrid llm-only</code>
Benchmarks	<code>poetry run ccop-eval evaluate run --model primus-reasoning --benchmarks B12 --benchmarks B13 --mode hybrid llm-only</code>
Full Suite Evaluation	<code>poetry run ccop-eval evaluate run --model primus-reasoning --benchmarks B13 --mode hybrid llm-only</code>
Teardown	<code>./src/scripts/start_local.sh --stop</code>

6 RAG Infrastructure Setup

Since the project targets deployment in isolated Critical Information Infrastructure (CII) environments — which are typically air-gapped or on-premise with restricted external connectivity — the RAG infrastructure was designed to operate with zero cloud dependencies. All components run locally via Docker, ensuring the pipeline can be deployed in any environment without requiring external API calls or managed services.

The local RAG stack consists of:

Component	Technology	Purpose
Vector database	Qdrant (Docker)	Hybrid search (dense + sparse) with RRF fusion
Dense embeddings	BAAI/bge-large-en-v1.5 via sentence-transformers	1024-dimensional semantic embeddings
Sparse embeddings	Qdrant/BM25 via FastEmbed	Keyword-based retrieval for exact terminology matching
Orchestration	LangGraph	Stateful adaptive RAG pipeline with self-correction loops
LLM inference	Ollama + Llama-Primus-Reasoning	Local model inference (4-bit quantized, GGUF)

A corpus of 8 authoritative CCoP documents was ingested into the pipeline, including the primary CCoP 2.0 Second Edition document, the Cybersecurity Act 2018, and CSA's Guidelines for Auditing Critical Information Infrastructure among others. These documents form the retrieval corpus from which the pipeline grounds LLM responses in authoritative regulatory text.

6.1 Retrieval Methodology

The retrieval pipeline processes each query through six stages before the LLM step:

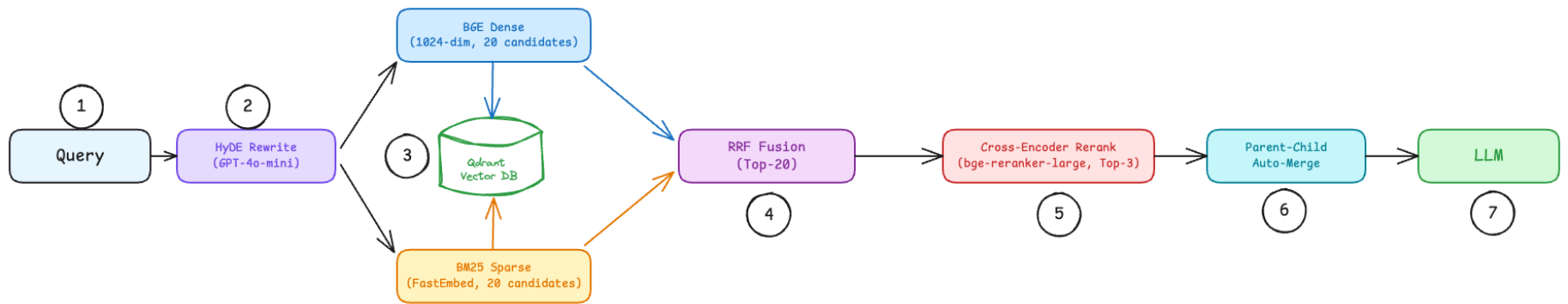


Figure 3: Retrieval pipeline — six stages from HyDE rewrite to parent-merge before the LLM step

1. **HyDE Rewrite** — the query is first expanded by a hypothetical-answer rewriter (GPT-4o-mini) that drafts a short response in CCoP-style regulatory language. The original query and the hypothetical answer are concatenated and used as the retrieval input, closing the vocabulary gap between lay phrasing (e.g., "can MFA be skipped?") and corpus phrasing ("multi-factor authentication exemption"). Please refer to Appendix 21.1.2.
2. **Dual Encoding** — the HyDE-rewritten query is encoded in parallel through two independent embedding models:
 - Dense: BGE-large-en-v1.5 (1024-dim, L2 normalized) — captures semantic meaning. A query about “access control” matches clauses discussing authentication and authorization even when the exact phrase is absent.
 - Sparse: BM25 via FastEmbed — captures exact terminology. Required for matching clause references (“5.1.5”), regulatory acronyms (“CIIO”), and domain-specific terms that dense embeddings generalize away.
3. **Parallel Prefetch** — each encoding retrieves its own candidate set from Qdrant independently: 20 dense candidates and 20 sparse candidates.
4. **Reciprocal Rank Fusion (RRF)** — the two ranked lists are merged into a single list of 20 candidates using RRF with equal weighting. RRF scores each document by summing $1/(k + \text{rank})$ across both lists, promoting documents that rank well in either or both retrieval paths without requiring score normalization between the two models.
5. **Cross-Encoder Reranking** — the 20 RRF candidates are re-scored by bge-reranker-large, which jointly encodes each query-document pair with full cross-attention. This resolves a limitation of bi-encoder retrieval: bi-encoders embed query and document independently, so semantically close clauses (e.g., 5.1.1 on access management vs 5.1.2 on authentication requirements) receive similar scores. The cross-encoder attends to both texts simultaneously, differentiating fine-grained regulatory distinctions. The top 3 are selected without applying any score threshold — relevance scores are logged for analysis but not used to filter, since gating before a baseline is established would mask retrieval quality issues.
6. **Parent–Child Auto-Merge** — when the top-3 reranked chunks include sibling sub-clauses (e.g., 5.1.1 and 5.1.2 both selected), they are merged into the parent clause (5.1). This prevents fragmented context where a single regulatory requirement spans adjacent sub-clauses and gives the LLM coherent passages instead of clipped sentences. The merged context is passed to the LLM for generation.

7 Evaluation Framework

The evaluation infrastructure uses two independent scoring mechanisms — RAGAs for automated retrieval-quality metrics, and an LLM-as-Judge with a custom 6-dimension rubric for reasoning, grounding, and citation correctness. The two are complementary: RAGAs scores retrieval and surface response quality; the LLM Judge scores compliance-reasoning correctness against the ground truth.

7.1 RAGAs Quality Metrics

RAGAs evaluates retrieval quality across three metrics, all applicable to RAG-augmented responses only (LLM-only mode does not produce retrieval contexts):

Metric	Bracket	What it measures
context_precision	Retrieval Quality	Whether retrieved chunks are relevant to the question
context_recall	Retrieval Quality	Whether retrieved chunks contain the information needed to answer
context_faithfulness	Model-RAG Grounding	Whether the response is supported by the retrieved context

The three metrics diagnose distinct retrieval failure modes: low precision indicates noise in the top-N; low recall indicates the right chunks were missed; low faithfulness indicates the model is not using what was retrieved.

7.2 LLM-as-Judge Scoring

The LLM Judge applies a single benchmark-agnostic rubric to every test case. Benchmark-specific signal is supplied through the ground-truth payload per test case (`expected_response`, `key_facts`, `clause_reference`, `forbidden_claims`, `hallucination_patterns`); the rubric itself does not change between benchmarks. This concentrates evaluation logic in a single auditable specification and ensures every score across the corpus uses the same scale.

Each response is scored on six independent dimensions, each on a 0–3 anchored scale (full anchor definitions in Appendix 21.2.1)

Dim	Name	What it measures
D1	verdict_accuracy	Whether the response's final verdict matches the expected answer
D2	justification_quality	Whether the reasoning is logically sound and internally consistent
D3	factual_grounding	Claim-level check — atomic claims supported by retrieved context, key_facts, or cited clause text
D4	scope_appropriateness	Whether the response stays within scenario constraints
D5	actionable_way_forward	Whether analysis is translated into specific feasible next steps
D6	citation_correctness	Citation-level check — clause IDs are real and accurately described

Decoupling D3 (claim-level) and D6 (citation-level) is the central innovation of the rubric. A response can cite real clause IDs whose substantive descriptions don't follow from those clauses (correct IDs, wrong descriptions) — that fails D6 but passes D3. The reverse also holds. Treating them as separate signals lets the judge distinguish citation hygiene from substantive grounding, both essential to compliance reasoning but failing in distinct ways. D3 and D6 are computed by count-based ratios over atomic units (claims for D3, citations for D6) rather than holistic judgement, reducing inter-judge variance on what would otherwise be the two most subjective dimensions.

Citation correctness is supported by a deterministic verification infrastructure that runs *before* the judge call: each citation in the response is classified as CORRECT, IMPRECISE, MISATTRIBUTED, FABRICATED, or EXTERNAL by checking against a doc-keyed clause inventory and a clause-text cache loaded from Qdrant at startup. The judge consumes these classifications as input rather than performing clause lookups itself, keeping it focused on reasoning evaluation rather than fact retrieval. Full taxonomy is in Appendix 21.2.3.

The judge runs against qwen/qwen3-235b-a22b-07-25 via OpenRouter at temperature 0.2 — a different model family from the system under test (Llama-Primus-Reasoning) to avoid self-evaluation bias. A secondary judge (gpt-4o-mini) is invoked only on measurement snapshots designated for human-review comparison. Runtime configuration, JSON parse retries, and the skip-and-flag policy for terminal failures are documented in Appendix 21.2.4.

8 Experiments & Ablation Studies

This section reports the iteration history that produced the production retrieval pipeline (Section 6) and the LLM Judge rubric (Section 7). Detailed experiment tables are in Appendix 21.3.

8.1 Retrieval Iteration — Headlines

The production retrieval stack documented in Section 5 is the result of 41 lab experiments. The architecture-changing experiments concentrate around three improvements: (a) chunk **contextualization** at indexing time (Exp #14) gave both the bi-encoder and cross-encoder regulatory anchors to lock onto on short queries; (b) **HyDE query rewriting** (Exp #17) bridged acronym expansion that bi-encoders cannot perform via attention alone; and (c) the **RRF ensemble** of dense rank and cross-encoder rank (Exp #28) preserved bi-encoder recall on cases where the cross-encoder under-weighted regulatory tokens. Six smaller iterations (TOC dot-leader filter, parent-child auto-merge, top_n tuning from 8 to 3, cross-encoder model upgrade from ms-marco-MiniLM-L12-v2 to bge-reranker-large, generation-prompt seed-phrase removal, structured ****Sources:**** footer) followed and were folded into the Exp #41 production migration. The full 15-row iteration table is in Appendix 21.3.1.

8.2 14.2 Judge Iteration — Headlines

The LLM Judge evolved through twelve iterations between the original 5-dimension cliff design and the production 6-dimension equal-weight rubric (Section 6). The two architecturally significant changes were (a) **discarding cliff weighting** in favour of equal weights so the composite became responsive to all dimensions, and (b) **splitting the original D3 (factual_grounding) into two separate dimensions** — claim-level grounding (D3) and citation-level correctness (D6) — so a response that cites real clauses with wrong descriptions is scored differently from one that grounds correctly but cites poorly. Reliability improvements followed: count-based ratios for D3 and D6 (replacing holistic 0–3 judgement), JSON parse retries with structured logging, and the load-bearing anti-leniency instruction in the system prompt. The full 12-row iteration table is in Appendix 21.3.2.

9 Evaluation Results

The system was evaluated on the 18-case stratified sample under both modes — llm-only (parametric only) and hybrid (RAG-augmented) — using the same model (Llama-Primus-Reasoning), same judge (Qwen3-235B-A22B), and same six-dimension rubric (Section 7.2).

9.1 Native vs RAG Evaluation — Six-Dimension Comparison

Per-dimension scores (mean across 18 cases):

Dim	LLM-only	Hybrid (RAG)	Impact Δ
D1 verdict_accuracy	0.278	0.370	+0.093
D2 justification_quality	0.593	0.611	+0.019
D3 factual_grounding	0.222	0.370	+0.148
D4 scope_appropriateness	0.593	0.852	+0.259
D5 actionable_way_forward	0.481	0.389	−0.093
D6 citation_correctness	0.185	0.278	+0.093
Overall (mean across dims)	0.392	0.478	+0.086

Insights:

- RAG's lift concentrates on the dimensions it should affect.** D1 (verdict), D3 (claim grounding), D6 (citation correctness), and D4 (scope) are the dims that depend on regulatory specifics. RAG lifts these by 9–26 pp. D2 (reasoning quality) is parametric-driven and barely moves. D5 (actionability) regresses slightly — the RAG-augmented model is more conservative about prescriptive next steps when retrieved context constrains it.
- D4 shows the largest gap (+0.26).** Scope appropriateness is where retrieval helps the model stay on the question rather than drift into adjacent regulatory topics. Without retrieved chunks anchoring the response, the LLM-only model frequently produces meta-analysis (*"the question pertains to..."*, *"I recall from training..."*) rather than a direct answer.

3. **D5 is the only regression** (−0.09). The LLM-only model produces longer, more prescriptive lists of generic remediation steps; the RAG-augmented model produces shorter responses tied to retrieved clauses. Length and prescriptiveness differ from correctness — the regression on D5 reflects the rubric rewarding actionable detail, not a quality drop.
4. **Citation correctness (D6) remains low even with retrieval (0.278)**. RAG improves the citation rate but doesn't eliminate misattribution — the model still occasionally cites real clauses without their substantive content matching the claim. This is a known limitation and a target for future improvement.

9.2 Evaluation Across Benchmarks

Per-benchmark composite scores on the post-audit 18-case stratified validation sample, both modes:

Benchmark	Name	LLM-only	Hybrid (RAG)	Impact Δ
B01	CCoP Applicability & Scope	0.44	0.61	+0.17
B02	Compliance Classification	0.11 ✗	0.44	+0.33
B03	Conditional Compliance Reasoning	0.28	0.28	0.00
B04	IT / OT Classification Boundary	0.78	0.44	−0.33
B05	Control Comprehension	0.22	0.28	+0.06
B06	Intent Understanding	0.28	0.22	−0.06
B07	Gap Identification Quality	0.50	0.50	0.00
B08	Risk-Based Prioritization	0.72	0.67	−0.06
B09	Risk Identification & Residual Risk	0.72	0.78	+0.06
B10	Risk Justification Coherence	0.56	0.50	−0.06
B12	Audit Perspective Alignment	0.33	0.50	+0.17
B13	Evidence Expectation Awareness	0.17	0.61	+0.44
B14	Remediation Quality & Feasibility	0.56	0.56	0.00
B18	Responsibility Attribution (SG)	0.11 ✗	0.89	+0.78

B21	Hallucination & Over-Specification	0.28	0.61	+0.33
B22	Waiver / Exception Reasoning	0.44	0.39	−0.06
B23	Multi-Regulator Coordination	0.33	0.17	−0.17
B24	Incident Response Guidance	0.22	0.17	−0.06

Insights:

1. **RAG strongly helps where regulatory specifics dominate the answer (4 benchmarks > +0.30 lift).** B18 (responsibility attribution, +0.78), B13 (evidence expectation awareness, +0.44), B02 (compliance classification, +0.33), B21 (hallucination resistance, +0.33). All four require precise CCoP-specific text that the model's training data alone cannot fully encode.
2. **RAG is approximately flat where parametric memory already covers the topic** (B07, B08, B09, B10). These benchmarks test general risk-and-compliance reasoning that the base model handles well from training. Retrieval neither helps nor hurts.
3. **Two LLM-only failures (below 15% score): B02 and B18.** Both are cases where the model has parametric knowledge of the topic concept (MFA, governance roles) but not the specific CCoP text. Without retrieval, the model fabricates or evades. With retrieval, B02 lifts to 0.44 and B18 to 0.89.
4. **RAG underperforms LLM-only on five benchmarks** (B04, B23, B24, plus marginal B05/B22). Two distinct causes: (a) **out-of-corpus content** — B23 (multi-regulator coordination across MAS/PDPA/IM8) and B24 (CII Regulations 2018 incident-reporting forms and timings) reference material not in the audited corpus; (b) **parametric strengths** — B04 (IT/OT classification) is well-covered by general cybersecurity training, and the retrieved context introduces noise that reduces precision. These benchmarks identify where retrieval-corpus expansion is the next-step intervention.

9.3 Worked Example — B13-001 (Evidence for Board Cybersecurity Oversight)

The contrast between Native (LLM-only) and RAG-augmented responses on this single test case illustrates the dominant Native-mode failure mode in regulatory reasoning: the model must guess which CCoP clauses apply, and when it guesses, it tends to either fabricate clause content or substitute generic best-practices for the regulatory specifics the question demands.

Question:

"What evidence should a Risk Manager prepare to demonstrate board cybersecurity oversight?"

Ground-truth verdict: A Risk Manager should prepare cybersecurity reports to the board (CCoP Clause 3.2.1(d)), risk register updates including residual-risk monitoring (Clause 3.2.5), board meeting minutes reflecting cybersecurity discussions, results from cybersecurity exercises (Clause 7.3.1), and remediation plans following exercises (Clause 7.3.5).

	Native (LLM-only)	RAG (Hybrid)
Raw model response	"Identifying Relevant Clauses. <i>From my training knowledge, I recall that CCoP 2.0 emphasizes the importance of effective communication between management and the board regarding cybersecurity risks. Specifically, Clause 5.2.1 discusses the need for regular reporting to the board on risk management practices and their effectiveness. Analyzing Evidence Requirements. To demonstrate board oversight, evidence should include: Risk Assessment Reports (comprehensive analyses of potential threats and vulnerabilities); Incident Response Plans (documentation showing preparedness for cyber incidents); Cybersecurity Policies (up-to-date policies that align with organizational objectives); Training Records (evidence of staff training on cybersecurity best practices).</i> Considering Alternative Approaches. While CCoP 2.0 focuses on the board's oversight role, <i>it may not specify exact types of evidence. However, industry best practices suggest that regular reporting, including metrics and key performance indicators (KPIs), can effectively demonstrate oversight. Potential Points of Failure.</i> Without direct references to specific clauses in CCoP 2.0, there is a risk of misinterpretation. It's important to ensure the evidence aligns with both regulatory requirements and industry standards..."	"Evidence for Board Cybersecurity Oversight: <i>To demonstrate board-level oversight of cybersecurity, a Risk Manager should prepare evidence that showcases the CHIO's compliance with regulatory requirements:</i> Cybersecurity Risk Management Framework — outlines the organization's approach to managing cybersecurity risks, including risk assessment methodologies, roles and responsibilities, communication plans, and reporting structures. Risk Register Updates — regular updates demonstrate ongoing monitoring of identified risks and their mitigation strategies, including evidence that residual risk thresholds are not breached, as per Clause 3.2.5 of CCoP 2.0. Board Meeting Minutes — should reflect discussions on cybersecurity matters, including reviews of risk assessments, incident reports, and updates from the Cyber Incident Response Team (CIRT). Cybersecurity Reports to the Board — regular, detailed reports provided by senior management to the board, as mandated in Clause 3.2.1(d) of CCoP 2.0, should be documented. Results from Cybersecurity Exercises — documentation of scenario-based exercises (as per Clause 7.3.1), including the testing of incident response plans, business continuity plans, disaster recovery plans, and crisis communication plans. Remediation Plans and Compliance Reports — submission of remediation plans following cybersecurity exercises (Clause 7.3.5) and compliance reports to the Commissioner..."
Verdict	Generic best-practices substitution; never identifies the specific board-oversight clauses (Clause 3.2.1(d), Clause 3.2.5, Clause 7.3.1, Clause 7.3.5)	Cites all board-oversight clauses with correct attribution

	Native (LLM-only)	RAG (Hybrid)
Hallucination 1: clause misattribution	"Clause 5.2.1 discusses the need for regular reporting to the board on risk management practices..." — fabrication . CCoP Clause 5.2.1 is about "accounts that have access to the CII" (account privileges and least-privilege rules), not board reporting. The correct clause is Clause 3.2.1(d) which the Native model never identifies.	Not present
Hedging on clause existence	"...it may not specify exact types of evidence. However, industry best practices suggest..." — concedes inability to find specifics, then substitutes generic content	Not present
Generic-substitution pattern	Lists generic items (risk reports, IRP, policies, training records) that any compliance framework would expect, without tying any to a CCoP clause	Each item tied to a specific cited clause
Citation grounding	"Clause 5.2.1" — real clause ID, wrong substantive content (MISATTRIBUTED in citation taxonomy, Section 7.2)	Clause 3.2.5, Clause 3.2.1(d), Clause 7.3.1, Clause 7.3.5 — all real, all correctly described
D1 verdict_accuracy	0/3	1/3
D3 factual_grounding	0/3	2/3
D6 citation_correctness	0/3	1/3
Outcome	FAILED (0.17)	PASSED (0.61)

What the worked example demonstrates:

The Native model knows that "board cybersecurity oversight" is a CCoP-relevant topic and that some clause must govern it. But it does not have the specific text of Clause 3.2.1(d), Clause 3.2.5, Clause 7.3.1, or Clause 7.3.5 in its parametric weights. Without those, it falls back on two failure modes: it **misattributes** content to a real clause it does have indexed (Clause 5.2.1, which exists but covers a different topic), and it **substitutes** generic best-practices content for the regulatory specifics the question demands. Both are dangerous in a compliance context, the first plants a false citation in audit trails, the second produces an answer that fails the regulatory specificity the question is testing.

The RAG-augmented response retrieves the actual board-oversight clauses and answers with each item tied to a real, correctly attributed clause. This is not because the model is "smarter" with retrieval but because retrieved context shifts the burden from "recall the right clause from training" to "use the clause text presented in context", which is a fundamentally easier task.

10 Observations

The five observations below distil the patterns visible in Section 9 (Evaluation Results) into claims about *where* and *why* retrieval-augmented generation succeeds or fails on regulatory-reasoning tasks. They are the load-bearing findings that motivate the next-step work (Section 19).

Observation 1 — RAG's contribution is dimensional and pattern-specific, not uniform. The +0.086 mean lift from LLM-only to Hybrid is the average of a sharply non-uniform per-dimension pattern. Retrieval lifts D1 (verdict accuracy, +0.09), D3 (claim grounding, +0.15), D4 (scope appropriateness, +0.26), and D6 (citation correctness, +0.09) — the four dimensions that depend on knowing which CCoP clause is the right one and what it actually says. Retrieval does not lift D2 (reasoning quality, +0.02), and it slightly regresses D5 (actionability, −0.09). The interpretation is straightforward: retrieval is a tool for *correctness against external authority*, not a tool for *reasoning quality*. Where the rubric measures regulatory specificity, RAG helps; where it measures the model's intrinsic reasoning or prescriptive style, RAG is irrelevant or modestly constraining.

Observation 2 — Llama-Primus-Reasoning's reasoning capability is solid; the gap is fact recall, not reasoning. The D2 stability across both modes (0.593 Native → 0.611 Hybrid, +0.02) is the strongest signal that the base model's reasoning is already adequate. The model constructs coherent multi-step regulatory arguments with or without retrieval; what it lacks is reliable parametric memory of which CCoP clause says what. The Section 9.3 worked example illustrates this directly: the Native response is internally well-structured (it correctly enumerates *why* board oversight evidence matters and *what kinds* of evidence would be relevant) but anchors that reasoning to a misattributed clause. The reasoning chain is sound; only the regulatory premise is wrong. This is consistent with Llama-Primus-Reasoning's design as a cybersecurity-specialized reasoning model — it reasons well over whatever facts it is given. RAG fills exactly the gap the model has: it changes *what* the model reasons over without altering *how* it reasons. Future fine-tuning is best targeted at narrow factual recall (CCoP clause-content alignment) rather than at reasoning capability, which is already adequate for this problem class.

Observation 3 — RAG helps most where the model knows the topic but not the specific clause. The four benchmarks with $> +0.30$ hybrid lift (B18 governance roles, B13 evidence for board oversight, B02 MFA classification, B21 hallucination resistance) share a structural property: the underlying topic is well-represented in the model's parametric training (the model "knows" what MFA is, what board oversight means, what governance attribution looks like in cybersecurity), but the specific CCoP clause text is not. In Native mode the model substitutes generic best-practices content for the regulatory specifics — sometimes by inventing wrong clause attributions (the Clause 5.2.1 misattribution in Section 9.3 is the canonical example), sometimes by hedging across multiple plausible answers, sometimes by listing generic items that any compliance framework would expect. With retrieval, the same model with the same prompt produces the right answer because the burden has shifted from *recall* to *read*.

Observation 4 — Native LLM-only mode is structurally hazardous for compliance reasoning, not merely suboptimal. Two of the 18 cases (B02-001, B18-001) scored below 15% under Native mode. Both involved confidently-asserted regulatory specifics — a fabricated three-factor MFA minimum, a misattributed Clause 5.2.1 board-reporting clause that an unsuspecting reader could mistake for accurate citations. In a compliance domain, *confidently wrong* is qualitatively worse than *acknowledged uncertainty*: a fabricated clause attribution plants a false reference in audit trails and remediation plans downstream. The structural hazard is not that Native mode produces lower scores but that it produces *plausible* lower-quality output that resists casual review. The case for retrieval-grounded responses is stronger when the alternative is producing this class of confident error.

Observation 5 — RAG underperforms only at corpus boundaries, not in retrieval mechanics. Five benchmarks (B04, B23, B24, marginal B05/B22) score lower under Hybrid than Native. Each has a clean explanation: B23 (multi-regulator coordination across MAS TRM / PDPA / IM8) and B24 (CII Regulations 2018 incident-reporting forms and timings) require regulatory text that is not in the audited seven-document corpus, so retrieval surfaces tangential CCoP clauses and dilutes the response; B04 (IT/OT classification) is a topic the base model's general cybersecurity training already covers well, so retrieval introduces noise without recall benefit. None of these regressions traces to a retrieval mechanics failure (chunking, embeddings, reranking, fusion). They are corpus-coverage failures. The implication for next-step work is that retrieval is the right architectural choice for this problem class — the pipeline is doing what it should — and the highest-leverage improvements are corpus expansion (ingesting external regulator documents, the CII Regulations, and supplementary CSA guidance) rather than retrieval-quality tuning.

10.1 Issues Encountered/ Blockers

A recently discovered challenge was the ability for our scoring methodology to differentiate between factual precision (fraction of model's claims that are present in ground truth) vs valid model reasoning. RAGAs penalized the model's own reasoning claims since they were not present in ground truth and instead marked them as hallucinated. This was the motivation for redesigning LLM-as-judge's role to evaluate model's reasoning depth and claim-level hallucination verification independently against retrieved contexts.

We continue to tune the scoring framework to obtain a fair assessment of the gains provided by RAG implementation and identify the real residual gaps in the model's reasoning and interpretation of CCoP, when augmented with retrieval capability (RAG). No blockers.

Sections 11 and onwards represent project updates from Term 3, reported at mid-term status. The Term-2 body (ground-truth expansion, the hybrid RAG pipeline, and the six-dimension LLM-Judge) remains the baseline for hybrid-RAG implementation.

11 GraphRAG Ablation Study to Improve Hybrid-RAG Performance

Term 2 quantified what hybrid (naive) RAG closes and, just as importantly, what it does not. Two residual gaps survived retrieval augmentation.

First, **citation correctness (dimension D6) stayed low even with retrieval, at 0.278**. Flat hybrid retrieval improves the *rate* of correct citation but does not eliminate misattribution: the model still cites a real clause whose substantive content does not match the claim. The Clause 5.2.1 misattribution in the Term-2 worked example (Section 9.3) is the canonical case.

Second, **cross-clause and multi-hop compliance answers are invisible to flat chunk retrieval**. A CCoP answer is frequently distributed across several documents. B01 applicability, for instance, turns on the CCoP digital-boundary definition, together with the Response-to-Feedback clarification at paragraph 2.2, together with the Cybersecurity Act designation at section 7. These three passages share almost no vocabulary, so retrieval by text similarity surfaces one of them and misses the connection between them.

Term 3 attacks these two residuals by deepening the retrieval layer with *structure*: linking clauses through an explicit, typed knowledge graph rather than an unstructured vector index. The central question of the term is empirical: **does GraphRAG improve on hybrid (naive) RAG, and on which dimensions?** To answer it cleanly, Term 3 is designed as a **controlled ablation study**. The model under evaluation (Llama-Primus-Reasoning), the independent judge (Qwen3-235B), and the ground-truth suite are all held fixed, and only the retrieval layer is changed across three settings: no retrieval, hybrid RAG, and GraphRAG. Because only one variable moves, any measured difference is attributable to the retrieval *structure* alone. The hypothesis is that a graph in which clauses bridge through shared typed concepts closes the citation-correctness and cross-clause residual that flat vector retrieval cannot, without regressing the dimensions hybrid RAG already handles well.

12 What is GraphRAG

Naive RAG, the Term-2 hybrid pipeline, retrieves a flat set of text chunks by vector and keyword similarity and hands them to the language model. It carries no notion of how one chunk *relates* to another, so an answer that requires joining facts across chunks or across documents depends on all the pieces happening to be lexically similar to the query. For regulation, they frequently are not.

GraphRAG first builds a **knowledge graph** from the corpus: concepts become nodes, and typed relationships become the edges between them. Retrieval then works by traversing that graph, so the unit of retrieval is a *connected structure* rather than an isolated chunk. This makes multi-hop and cross-document reasoning a first-class capability: two clauses in different documents that share a concept are reachable through that shared node even when their wording is completely different.

The Term-3 design draws its GraphRAG blueprint from the **OMD-GraphRAG** paper¹ (Wang, Huang, Ge, Su, Liu and Lian, *Enhancing GraphRAG with Ontology-Guided Extraction, Multi-Dimensional Clustering and Dual-Channel Fusion*, China Unicom, arXiv 2603.25152v3, 2026). That paper extends the widely used open-source GraphRAG approach and reports a **9.21% average-F1 improvement** over a leading baseline (LightRAG) on a public multi-hop question-answering benchmark. It contributes three innovations. The first and third map directly onto compliance reasoning; the second is deferred as low value for our clause-level factoid ground truth.

1. **Ontology-guided knowledge extraction.** A predefined schema constrains what the extractor may produce and discards type-invalid facts. The paper reports this alone improves retrieval accuracy by 3.17%. This is discussed in Section 13.
2. **Multi-dimensional community clustering.** Richer thematic summaries for broad, global questions (3.43%). This is deferred in our build, since our questions are clause-level rather than thematic.
3. **Dual-channel retrieval fusion.** A graph channel and a thematic channel, combined with a traditional text-similarity channel and re-ordered by a cross-encoder reranker (3.32%). This is discussed in Section 16.

Two further papers frame the design. **GraphCompliance** (Chung and colleagues, *Aligning Policy and Context Graphs for LLM-Based Regulatory Compliance*, accepted at The Web Conference 2026) supplies an important cautionary finding: a graph used *only* to retrieve, without deeper structural reasoning, can actually underperform ordinary RAG (47.5 versus 49.5 micro-F1 in their study). The benefit comes from structure and reasoning, not from

merely attaching a graph to the same ranking. A third paper, *An Ontology-Driven Graph RAG for Legal Norms* (2025), motivates treating the clause hierarchy as the primary backbone of the graph. The full research trajectory through these papers is recorded in Section 17.3.

13 Ontology-Guided Knowledge Graph

Typical GraphRAG extracts facts by **schema-free prompting**: the language model is simply asked to find the entities and relations in the text, with no constraint on which types are allowed. On regulatory text this fails in a specific, observed way. An earlier Term-3 baseline built exactly such an un-governed graph and found that it modelled **the scenario narrative rather than the regulation**. It produced no concept for a clause, a control, or an obligation; it dumped whole clause sentences in as free-text nodes; and it generated duplicate and mistyped entities. A graph like that cannot bridge B01 across three documents, because the phrase “the CII’s digital boundary” never resolves to a single, reusable concept.

Ontology-guided extraction fixes this by defining, in advance, a formal schema with three parts:

- A fixed set of permissible **entity types**, such as *CII*, *CIIO*, *Digital Boundary*, *Password*, *Security Control*, *Regulator*, and *Obligation*.
- A fixed set of allowable **relationship types**, such as *applies to*, *within boundary*, *determined by*, *attribute of*, *mandates*, and *defers to*.
- A **type-constraint rule** for every relationship, specifying which entity types may sit on each side of it. A candidate fact is kept only if both endpoints have the correct type; otherwise it is discarded after extraction.

Why this works better for compliance, in concrete terms:

- **Every mention of a concept collapses to one canonical node.** “CII”, “critical information infrastructure”, and “the designated system” all become the single *CII* node. That is precisely what turns *CII* into a hub that bridges documents. Schema-free extraction instead scatters these into many near-duplicate free-text nodes that never connect.
- **The type-constraint rule suppresses the noise that pollutes flat retrieval.** A fact that would type-check to nonsense, such as an audit being the thing that “mandates” something, is dropped rather than stored, so the graph carries only regulation-shaped structure.

- **Domain law is encoded as explicit rules.** Our ontology asserts, for example, that a system cannot be classified as both IT and OT at once; it records thirty “is-a” hierarchy relationships; and it captures the distinction between mandatory (“shall”) and recommended (“should”) obligations. These are exactly the distinctions the Term-2 judge measured and that the base model frequently got wrong.

The domain ontology graph was an LLM-assisted iterative process. The ontology schema consisting of the type-constraint rules was defined by the human. The LLM was used to extract the entities and relationships directly from the source corpus of CCoP 2.0 and supporting documents. A second LLM pass was done over the corpus to critic and refine the first set of entities and relationships extracted and constrain them to the ontology schema. This helped to ensure we had a holistic entity-relationship mapping that adequately covered the entire corpus. Human review was essential to scrutinize the LLM extracted ERs (Entities and Relationships) based on relevance and real-world fit.

Our one deliberate departure from the paper concerns *which model does the extraction*. The paper drives extraction with a general-purpose language model following the schema prompt. Our trial of a lightweight model (GPT-4o-mini) under the same schema proved too weak for regulatory text: it dumped free text, mistyped entities, and varied from run to run. Since the ontology itself was authored to a high standard, the graph is built to match: **Claude Opus reads each clause and writes the typed facts, and the software only validates them against the schema and stores them.** Extraction is therefore a one-time cost, amortised across every future query. The result is a fully governed graph: all 869 clauses extracted, 1,935 typed facts, none violating the schema, resolving to 122 canonical concepts.

14 GraphRAG System Architecture

GraphRAG is added as a **new, self-contained retrieval path**. The Term-2 hybrid pipeline is left completely untouched and remains the default, honouring the project rule that any new retrieval mode must be strictly additive and preserve backward compatibility. A user selects the GraphRAG path with a single mode option; every existing mode behaves exactly as before.

Indexing (performed once). A build pipeline turns the seven source documents into the knowledge graph through the following stages.

Stage	What it does
Re-parsing and cleaning	Re-reads each source PDF from scratch and segments it into 869 clean clauses (nothing is reused from the Term-2 index)

Ontology specification	The locked schema: roughly 123 entity types, 64 relationship types, the type-constraint rules, the “is-a” hierarchy, the IT/OT disjointness rule, and obligation modality
Governed extraction	Opus-authored facts are validated against the schema and stored, one clause at a time, so the build is resumable
Enrichment passes	Additional passes add mandatory-versus-recommended modality (248 clauses) and split broad umbrella concepts into specific leaf concepts
Graph loading	The clause nodes, concept nodes, and their relationships are loaded into a graph database under a versioned, removable build tag
Retrieval aids	Concept rarity weights, a semantic (dense-vector) index over clause text, and 68 glossary definition nodes are computed to support retrieval

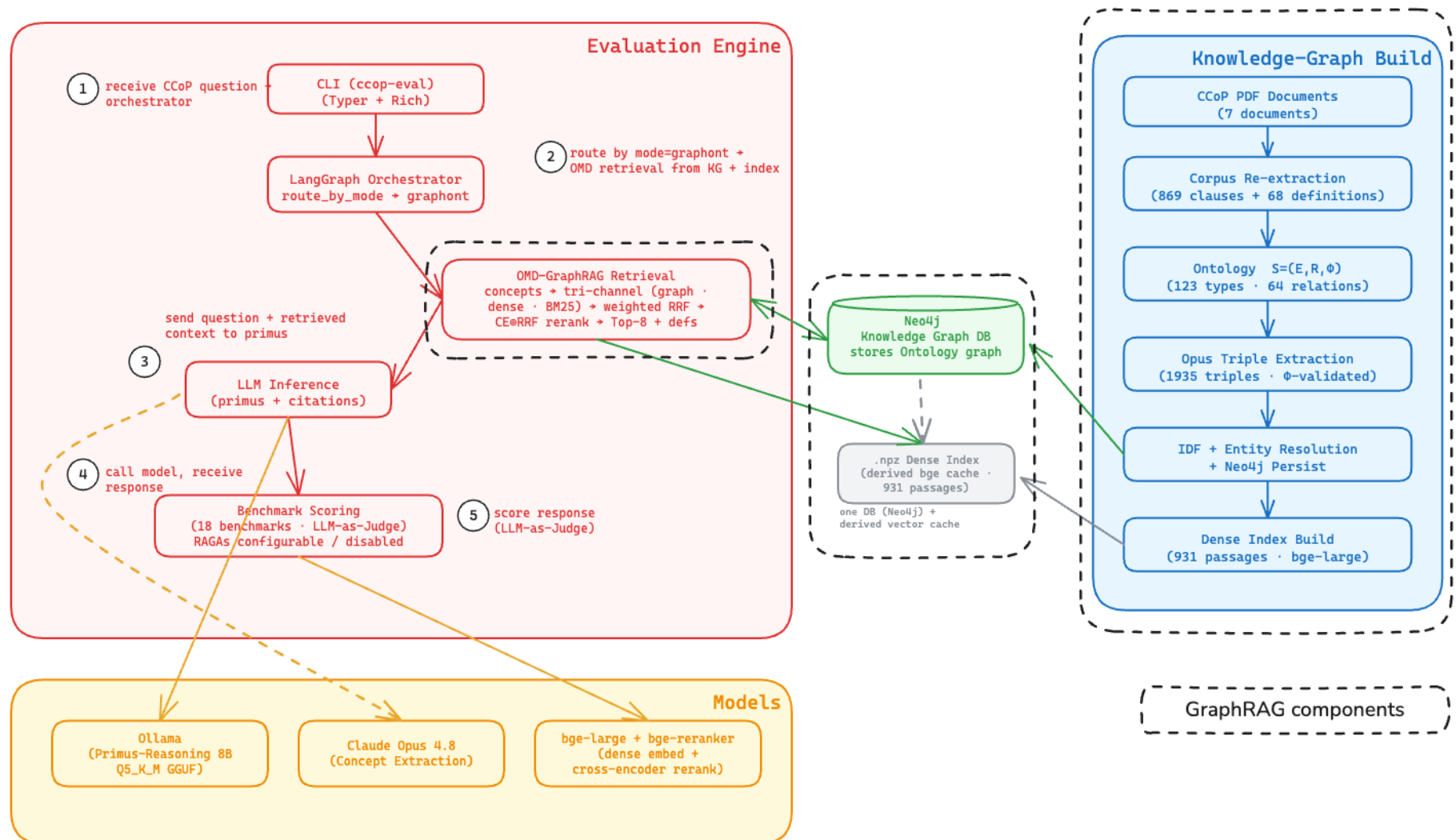


Figure 4: Container diagram for our GraphRAG implementation

A new store. A local **graph database (Neo4j)** joins the existing Qdrant vector store. It holds the knowledge graph and is consulted only by the GraphRAG path.

A new retrieval step. A standalone GraphRAG retrieval component reads only the knowledge graph and touches no existing part of the pipeline. When the GraphRAG mode is selected, the query is routed to this component, which assembles the grounding context in the same shape used by the hybrid pipeline. The downstream answer-generation step, in which the model reasons over the retrieved clauses and cites them, is therefore identical across all modes. This is what keeps the ablation comparison fair.

Backward compatibility. The integration is confined to a few well-defined connection points and adds the GraphRAG mode without modifying any existing mode, retrieval step, or data store. The GraphRAG path is strictly opt-in.

15 Domain Ontology Construction

Construction proceeds clause by clause. Opus reads a clause, writes the typed facts it contains, the software validates each fact against the schema, and the graph loader records the links between clauses and the concepts they invoke, as well as the links between concepts. The finished graph contains 863 clause nodes, 122 concept nodes, and 68 glossary-definition nodes, joined by 3,135 clause-to-concept links and 1,935 concept-to-concept relationships, with no fact violating the schema.

The schema itself is the formal ontology, written as three parts: the entity types (E), the relationship types (R), and the type-constraint rules (Φ). The three subsections below describe how each part was defined, what it contains, and how the finished graph is populated against it.

15.1 Entity Types (E)

An entity type is a *kind of thing* the regulation talks about, at the level of a type rather than an individual item. This is a deliberate design choice: a domain controller, a DNS server, and an operational-technology historian are all treated as the single type *Server*, and the specific make or model is recorded as a property of the node rather than as a new type. This keeps the vocabulary compact and reusable, which is what allows different clauses to converge on the same concept.

The entity types were identified by hand, by reading the entire corpus in two passes. The first pass worked top-down from the structure of the Code of Practice (its chapters on scope, audit, governance, asset identification, protection, detection, response, resilience, training, and operational technology)

together with the distinctions the eighteen benchmarks require. The second pass worked bottom-up from every glossary term and from the subject and object of every obligation clause, since these nouns are exactly the things the graph must be able to link.

This produced **122 entity types, organised into nine categories**. A single abstract parent type, *Actor*, sits above all duty-bearing roles, and 37 “is-a” relationships record the hierarchy among types (for example, a Chief Information Security Officer is an Organizational Role, which is an Actor; a Cloud Service Provider is a Third Party).

Category	Types	Representative entity types
Systems, assets and boundaries	20	Computer System, CII, CII Asset, IT System, OT System, Enterprise Network, Digital Boundary, Essential Service
Processes and activities	20	Risk Assessment, Threat Modelling, Penetration Testing, Audit, Incident Management, Monitoring
Actors and roles	18	CIIO, Board, Chief Information Security Officer, Auditor, Third Party, Incident Response Team
Documents, plans and artifacts	14	Policy, Risk Register, Incident Response Plan, Audit Report, Remediation Plan
Security controls	13	Security Control, Access Control Mechanism, Cryptography, Network Control, Security Configuration Baseline
Regulatory and legal	12	Regulator, Legislation, Code of Practice, Provision, Obligation, Waiver, Designation
Risk and threat concepts	11	Cybersecurity Risk, Residual Risk, Cybersecurity Threat, Vulnerability, Likelihood, Impact
Security objects and data	8	Account, Privileged Account, Password, Password Length, Default Credential, Hash Storage
Governance and compliance	6	Design Principle, Compliance Status, Condition, Compliance Gap, Deadline
Total	122	

The most heavily invoked types are the ones at the center of the compliance frame: the CIIO (Critical Information & Infrastructure Owner) appears in 382 clauses, the generic Provision in 328, and the CII itself in 296.

15.2 Relationship Types (R)

A relationship type is a *typed predicate* that connects two entity types, capturing the verbs and duties the clauses express. Relationships were drawn from the same two-pass reading of the corpus: each obligation or statement of fact in a clause becomes one or more relationships between the entities it names.

This produced **72 relationship types**, of which **71 are actually used** in the finished graph, across **1,935 relationships** in total. They fall into a handful of functional families.

Family	Purpose	Representative relationships
Applicability and scope	Fix what the regulation covers and where its boundary lies	<i>applies-to, delivers, in-sector, within-boundary, determined-by, designates, connected-to, excluded-from-scope, classified-as</i>
Obligation and modality	Distinguish mandatory from recommended duties and their conditions	<i>mandates, recommends, conditioned-on, has-obligation, has-deadline</i>
Control and protection	Link controls to what they protect or mitigate	<i>implements, protects, mitigates, detects, applies-principle</i>
Threat and risk	Connect threats, vulnerabilities and the activities that address them	<i>identifies, targets, addresses, responds-to</i>
Accountability and governance	Assign duties and reporting lines across roles	<i>delegates-to, responsible-for, reports, conducts, has-certification</i>
Incident and response	Capture the response lifecycle	<i>activates, recovers, remediated-by</i>
Limitation and deferral	Record what the regulation does <i>not</i> specify	<i>does-not-specify, defers-to, excluded-from-scope</i>

The most frequent relationships reflect the regulation’s emphasis on controls and duties: *implements* appears 248 times, *mandates* 200, *protects* 196, *mitigates* 85, and *identifies* 82. The limitation-and-deferral family is small but important, because it is what lets the graph represent the absence of a requirement (for example, that the Code does not specify a password length), which is central to the hallucination-resistance benchmarks.

15.3 Type-Constraint Rules (Φ)

Φ is what makes the extraction *governed* rather than free-form. It assigns to every one of the 72 relationships a permitted **domain** (the entity types allowed on the subject side) and a permitted **range** (the entity types allowed on the object side). A candidate fact is admitted into the graph only if the subject's type is in the relationship's domain and the object's type is in its range; any fact that fails this check is discarded after extraction. A few example rules:

Relationship	Permitted subject types (domain)	Permitted object types (range)
applies-to	Provision, Code of Practice	CII, CIIO, IT System, OT System
delivers	CII, Computer System	Essential Service
within-boundary	CII, CII Asset, Computer System	Digital Boundary
determined-by	Digital Boundary	Regulator
classified-as	CII	IT System, OT System

The constraints are not one-to-one: 34 of the relationships permit more than one subject type and 38 permit more than one object type, which is what lets a single relationship like *applies-to* connect the regulation to several kinds of governed system while still rejecting nonsensical pairings. Φ is complemented by one hard **disjointness rule**: a system cannot be classified as both IT and OT at the same time, which is the distinction the IT/OT boundary benchmark depends on.

Because every fact passes through Φ at construction time, the finished graph is fully type-consistent: all 869 clauses were processed and none of the 1,935 stored relationships violates the schema. This is the concrete difference from schema-free GraphRAG, where no such check exists and the noise it admits is what pollutes retrieval.

15.4 Clause Nodes, Linked to Concept Graph

The concept layer so far is only half of the graph. The other half is the regulation itself: **every clause is also stored as its own node**, linked to the concepts it invokes. This join connects the regulatory text to the concept structure and is what lets the system retrieve the *right clauses* for a question. There are **863 clause nodes**, one per clause, spanning the seven source documents:

Source document	Clause nodes
CCoP 2.0	379
CCoP Response to Feedback	280
Security By Design	98
Cybersecurity Act 2018	62
Auditing Guidelines	17
Risk Assessment Guide	15
Threat Modelling Guide	12
Total	863

Each clause node carries its citation identifier (for example, “CCoP 2.0, clause 5.9.2(b)”), its source document, and its **verbatim text**, since it is this exact text that grounds and is cited by the model, so the graph never paraphrases the regulation. Each node is then linked to the concepts appearing in it by an **invokes** relationship, giving **3,135 links**: 840 of the 863 clauses invoke at least one concept (3.73 on average), and the remaining 23 are administrative or header clauses left unlinked rather than padded with noise. A glossary layer completes the picture, with 68 definition nodes attaching authoritative text to the concepts they define through 28 links.

The result is a **two-layer graph**: a text layer of clause nodes holding verbatim regulation, and a meaning layer of concepts joined by the typed relationships of Section 15.2, stitched together by the invokes links. This is what enables clause retrieval: a question is mapped to the concepts it concerns,

and the graph returns the clauses that invoke them. Because concepts are shared, a single concept becomes a hub that gathers clauses across documents. The CII concept, for instance, is invoked by 296 clauses spanning all seven documents, so a scope question anchored on the CII reaches the CCoP definition, the Act’s designation, and the Response-to-Feedback clarification together, even though their wording differs. Section 16 describes the retrieval procedure that walks this structure.

15.5 Cross-Document Bridges

The graph’s value appears when typed facts combine: a concept that several clauses invoke becomes a bridge across documents. The two anchor benchmarks illustrate this, using the actual extracted facts.

B01-001, applicability and scope.

A healthcare provider’s patient monitoring systems and MRI machines have been designated as CII. Their hospital administration system (patient records, billing, appointments) shares the same enterprise network. Does CCoP 2.0 mandatory compliance extend to the hospital administration system?

The expected answer is *not applicable*: compliance follows the CII’s digital boundary as determined by the regulator, not another system merely sharing the same enterprise network. That answer is distributed across three documents, which the graph links through the shared concepts *CII* and *Regulator*.

Source clause (document)	Facts extracted into the graph
CCoP 2.0, clause 1.4.1	the Code of Practice <i>applies to</i> the CII; the Code of Practice <i>applies to</i> the CIIO
Response-to-Feedback, paragraph 2.2	the Code of Practice <i>applies to</i> the CII; the CII is <i>within</i> the Digital Boundary; the Digital Boundary is <i>determined by</i> the Regulator
Cybersecurity Act, section 7	the Regulator <i>designates</i> a Computer System; that Computer System <i>delivers</i> an Essential Service

Though the three clauses share almost no wording, they form one connected structure: clause 1.4.1 and paragraph 2.2 both point to *CII*, while paragraph 2.2 and section 7 both point to *Regulator*. The decisive scope fact, that the boundary is the CII’s digital boundary as determined by the regulator and not the enterprise-network perimeter, is exactly the path *CII* → *within* → *Digital Boundary* → *determined by* → *Regulator*, which a flat index cannot represent.



Figure 6: Graphical representation of key entity and clause nodes for B01-001 with their relationships

B05-001, control comprehension (password requirements).

What are the minimum password requirements according to CCoP 2.0?

The expected answer is that CCoP 2.0 specifies *no* password length or complexity requirement and defers to industry standards such as NIST; its only password obligations are that default passwords be changed and passwords stored in hashed form. This is a common hallucination trap, since a model is tempted to invent a minimum length. The answer bridges two documents through the shared concept *Password*.

Source clause (document)	Facts extracted into the graph
CCoP 2.0, clause 5.9.2(b)	a Password is <i>stored as</i> a hash; the Security Configuration Baseline requires default credentials to be <i>changed</i>
Response-to-Feedback, paragraph 11.28	Password Length is an <i>attribute of</i> Password; the Code of Practice <i>does not specify</i> a Password Length; the CIIO <i>defers to</i> an external standard

The control clause and the clarification are joined by the shared *Password* concept, so the fact that the Code defers on password length, the correct answer and the trap, is reachable from the control clause through that bridge.

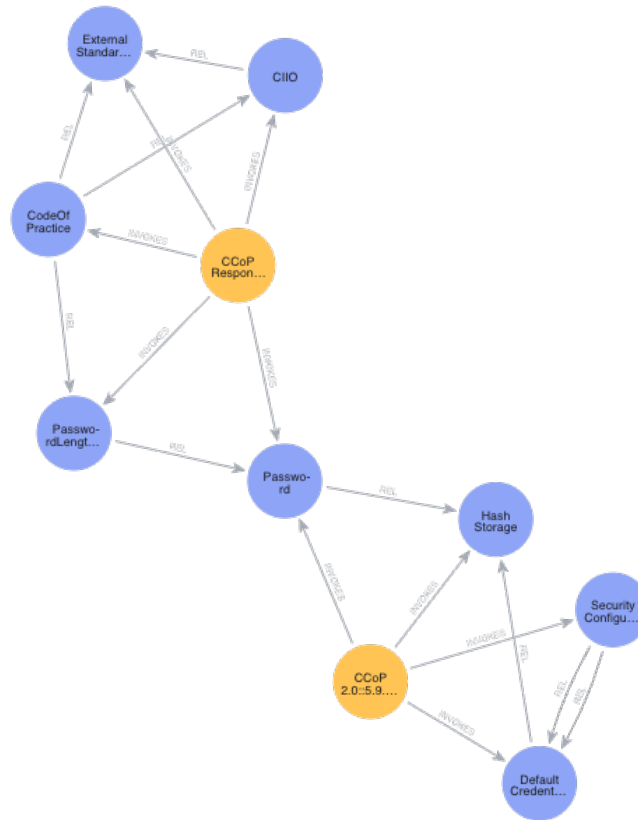


Figure 7: Graphical representation of key entity and clause nodes for B05-001 with their relationships

16 GraphRAG Retrieval Methodology

Retrieval turns a question into a small set of the most relevant clauses, handed to the model as grounding, in four stages: translate the question into graph

concepts (16.1), expand those concepts across the graph (16.2), recall candidate clauses through three parallel channels (16.3), and fuse and re-rank them into the final list (16.4). Known limitations follow in 16.5. Each stage corresponds

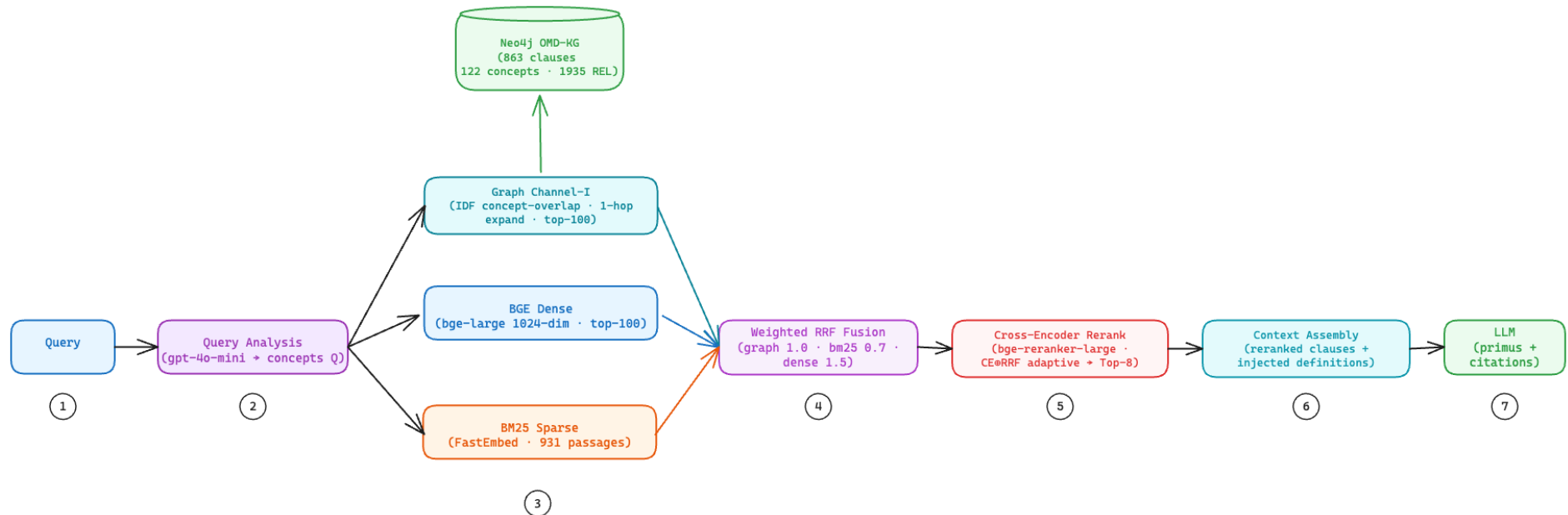


Figure 8: GraphRAG retrieval methodology- from query analysis, multi-channel recall, RRF fusion and reranking using cross-encoder

Each numbered step in the above diagram corresponds with the subsection explanation below.

16.1 Stage 1- User Query Translation

This first stage is what makes the method GraphRAG rather than ordinary search. The question is not embedded and matched against clause text directly; it is first **translated into the graph's concept vocabulary**, the same canonical concepts the clauses invoke (Section 15), so that question and clauses meet in one shared conceptual space and a relevant clause can be found even when it shares no words with the question.

The translation is done by a **language model acting as a constrained classifier**: a lightweight model (GPT-4o-mini, at temperature zero) is shown the full list of the graph's 122 concept names and returns only those, chosen word-for-word from the list, that the question is *about*. Because it may pick only from a fixed vocabulary, this is bounded selection rather than open-ended generation, so a small model is reliable here, unlike the graph *construction* step which required Claude Opus (Section 13). It selects by meaning rather than keyword (for example, "patient monitoring systems" and "MRI machines" are read as a *Computer System* and a *CII Asset*, not the *Monitoring* control), it surfaces implicit concepts the question depends on but never names (scope questions always surface *Digital Boundary*, *Obligation*, and, when contested, *Regulator*), and it prefers specific concepts over broad hubs, returning a focused two to six. For the anchor questions of Section 15.5 it yields *Digital Boundary*, *CII*, *Obligation*, and *Regulator* for B01-001, and *Password* and *Password Length* for B05-001.

16.2 Stage 2- Traversing the Graph

The selected concepts are extended by one step along the concept relationships, so clauses invoking a closely related concept are also considered. The expansion is gated by concept rarity: only specific, informative concepts expand, while a broad hub such as CII (which touches roughly a third of the graph) counts as a direct match but does not pull in neighbours, preventing it from flooding the candidate set.

16.3 Stage 3- Parallel Recall Channels

The question and its concepts are run through three complementary channels, each returning its own ranked list of candidate clauses, adapting the reference paper's dual-channel design so that a clause missed by one channel can still be found by another.

- Graph channel (structural): scores each clause by how many query concepts it invokes, weighted by concept rarity, so a rare decisive concept outweighs a common one and a focused answer outranks a generic clause that merely shares common concepts.
- Keyword channel: a classical keyword search over verbatim clause text, strongest on exact terminology and clause numbers.
- Semantic channel: a dense-vector similarity search over clause text, catching abstract clauses that concept overlap and keyword match both miss.

16.4 Stage 4- Weighted Rank Fusion

The three ranked lists are merged into one by weighted Reciprocal Rank Fusion, which combines candidates by their position in each list rather than by raw score, so no cross-channel score normalisation is required. Each channel carries a weight, the semantic channel highest, the graph channel next, the keyword channel least, following the project's earlier retrieval findings. A clause ranked well by any single channel still enters the merged pool, while a clause ranked well by more than one rises toward the top.

16.5 Stage 5- Cross-Encoder Reranking and Grounding

A cross-encoder reranker then reads each question-and-clause pair in the merged pool and scores their relevance jointly, producing a sharper ordering than fusion by position alone. Its ranking is combined with the fused ranking rather than used on its own, because the current reranker gives undifferentiated scores on very short questions; combining lets a confident reranker dominate while falling back on the fused order when it has no signal.

A final glossary step then attaches authoritative definitions of the query concepts as extra grounding, independent of the ranking. The top clauses, in verbatim text with their definitions, form the grounding context, assembled in the same shape the hybrid pipeline uses so that answer generation is identical across modes (as shown in Steps 6 and 7 in Figure 8)

16.6 Observed Limitations & Challenges

Two ranking issues remain before the mode is finalized: the rank fusion can dilute a result that is strong in a single channel but weak in the others, and the cross-encoder gives weak, clustered scores on very short factoid questions. Both point to the reranker as the next lever to improve, whether by adapting its influence to its own confidence or by adopting the stronger reranker used in the reference paper.

17 GraphRAG Evaluation

The GraphRAG methodology is built, is now wired into the evaluation harness, and is validated end-to-end on the two anchor benchmarks, B01 and B05, chosen because each exercises a distinct graph capability: B01 a three-document scope bridge, B05 a two-document control-and-clarification bridge. The worked bridges (Section 15.5) and the first result (Section 17.1) confirm that the graph recalls the distributed answer that flat retrieval misses.

Note that this is **not yet a full-suite result** as we continue to analyze results across a stratified subset of test cases to improve citation accuracy and model responses.

17.1 Hybrid vs GraphRAG: E2E Evaluation Result for B05-001

The GraphRAG mode has now been run end-to-end through the evaluation harness, and the first head-to-head result on the B05-001 anchor case gives a concrete, single-case illustration of the thesis of Section 11. Both modes used the same model, the same judge, and the same six-dimension rubric; only the retrieval layer differed.

The ground truth for this case is as follows.

- **Question:** *What are the minimum password requirements according to CCoP 2.0?*
- **Expected answer:** *CCoP 2.0 specifies no password length or complexity requirement. Per Response-to-Feedback paragraph 11.28, CSA defers to industry standards such as NIST for the appropriate password length. The only password obligations, in CCoP 2.0 clause 5.9.2(b), are that default passwords be changed and that passwords be stored in hashed form.*
- **Expected clause citations:** *Response-to-Feedback paragraph 11.28, and CCoP 2.0 clause 5.9.2(b).*

The two modes behaved as follows.

Aspect	Hybrid (naive RAG)	GraphRAG
Clauses retrieved	3	8
Expected clauses	0 of 2 (returned clauses 5.1.2, 5.6.1 and Response-to-	2 of 2 (both paragraph 11.28 and clause 5.9.2(b) retrieved,

retrieved	Feedback section 6, none of which govern passwords)	alongside the 5.9.2 parent)
Raw model response (verbatim)	<i>Minimum Password Requirements in CCoP 2.0</i> CCoP 2.0 does not explicitly outline specific minimum password requirements such as length, complexity, or frequency of changes. The code focuses on broader authorization and authentication controls (Clause 5.1.2) and network access control rules (Clause 5.6.1), but it does not specify detailed technical specifications for passwords. The emphasis is on implementing measures commensurate with the cybersecurity risk profile rather than prescribing exact password policies. Therefore, CIOs are expected to assess their specific risks and implement authentication controls that align with these assessments, without explicit minimum requirements provided by CCoP 2.0. Sources: CCoP 2.0 Clause 5.1.2; CCoP Response to Feedback Section 6	<i>Minimum password requirements according to CCoP 2.0 are not explicitly stated. The regulatory corpus does not specify a minimum password length or complexity for passwords or passphrases under the Cybersecurity Code of Practice (CCoP) 2.0. While it emphasizes the importance of managing passwords and passphrases, including changing default passwords and storing them in hashed forms, it does not provide specific requirements regarding their length.</i> Sources: CCoP Response to Feedback 11.28; CCoP 2.0 5.9.2(b)
Citations the model gave	Clause 5.1.2 and Response-to-Feedback section 6, neither of which is a governing clause	Paragraph 11.28 and clause 5.9.2(b), both of them the expected clauses
Were citations used correctly?	No	Yes
LLM Judge Score	0.39	0.78

Both models reached the same surface verdict, that no minimum is specified, but only GraphRAG grounded that verdict in the two clauses that actually govern it. Hybrid retrieved none of the governing clauses (its three chunks concerned access control and network access, not passwords), so its answer omitted the deferral to NIST and the change-default and hashed-storage obligations, and it cited clauses that do not contain the answer. This is exactly the Term-2 failure mode: a plausible answer resting on wrong citations, which is what the citation-correctness dimension (D6) is designed to catch, and which cost hybrid a zero there. GraphRAG’s concept bridge on *Password* surfaced both paragraph 11.28 and clause 5.9.2(b), lifting citation correctness from 0 to

2 and verdict accuracy from 1 to 3, and the composite from 0.39 to 0.78. This is a single case, on the benchmark the graph was designed around, and the reranker’s scores were tightly clustered (a known limitation, Section 16.5); it illustrates the mechanism rather than establishing a suite-level claim. The full ablation of Section 17.4 remains the real test.

17.2 Hybrid vs GraphRAG: E2E Evaluation Result for B01-001

The second head-to-head, on the B01-001 scope case, is a sharper illustration still, because here the two modes disagree on the answer itself. The setup is again identical across modes except for retrieval.

The ground truth for this case is as follows.

- **Question:** *A healthcare provider’s patient monitoring systems and MRI machines have been designated as CII. Their hospital administration system (patient records, billing, appointments) shares the same enterprise network. Does CCoP 2.0 mandatory compliance extend to the hospital administration system?*
- **Expected answer (label: not applicable):** compliance applies to the designated CII and its digital boundary, not automatically to every system on the same enterprise network, so the hospital administration system is not in scope merely for sharing the network. The digital boundary is jointly determined by CSA, the CIIO, and the Sector Lead, and is not the enterprise-network perimeter.
- **Expected clause citations:** CCoP 2.0 clauses 1.2.1 and 1.4.1, and Cybersecurity Act section 7, with Response-to-Feedback paragraph 2.2 as the decisive digital-boundary clause.

The two modes behaved as follows.

Aspect	Hybrid (naive RAG)	GraphRAG
Clauses retrieved	3	9
Expected clauses	None (returned clauses 3.3.1, 5.12.1 and 5.6.1, on	The decisive digital-boundary clause, Response-to-Feedback paragraph 2.2, plus the surrounding scope cluster (paragraphs 2.1, 2.5,

retrieved	governance and network controls, none about scope)	2.11)
Verdict reached	Wrong: compliance <i>does</i> extend to the administration system	Correct: compliance does <i>not</i> extend to it
Raw model response (verbatim)	<i>Compliance with CCoP 2.0 extends to all systems that are part of the Critical Information Infrastructure (CII). CCoP 2.0 mandates that CIIOs protect and manage cybersecurity risks across their entire designated CII, which includes both patient monitoring systems and MRI machines as well as the hospital administration system. The regulatory framework emphasizes comprehensive coverage of all components integral to the CII's operation. Key Points: Definition of CII, CCoP 2.0 defines CII broadly to encompass any information infrastructure essential for national security, public safety, or economic well-being; Scope Inclusion, the administration system, being part of the same enterprise network as the designated systems, falls under the regulatory purview due to its interconnectedness and potential impact on critical services; Comprehensive Protection, CIIOs must safeguard all aspects of their CII, including systems not explicitly listed if they contribute to its functioning or security posture. Conclusion: given the interconnected nature of modern healthcare IT environments and the potential cascading effects on critical services, compliance with CCoP 2.0 is expected to cover all systems integral to the CII.</i> <i>Sources: CCoP 2.0 3.3.1(a), 5.6.1, 5.12.1; Cybersecurity Act 2018 Section 11(7)</i>	<i>CCoP 2.0 does not explicitly mandate compliance to the hospital administration system if it is part of the enterprise network separate from the designated CII systems (the patient monitoring and MRI machines). The regulatory corpus indicates that CCoP 2.0 applies specifically to the CII systems, defined as an infocomm or operational technology system or network infrastructure vital to the continuous delivery of Essential Services (Security By Design Annex C). The CII includes computer systems within its digital boundary as defined by CSA, CIIOs, and Sector Leads (Response-to-Feedback 2.2). While the hospital administration system shares the same enterprise network with the MRI machines and patient monitoring systems, there is no explicit requirement in CCoP 2.0 that extends compliance beyond the designated CII boundaries. Clauses such as 10.2.1 emphasize physical segregation between OT CII networks and enterprise networks, suggesting a focus on isolating critical infrastructure from other systems. Therefore, based on the regulatory corpus, compliance with CCoP 2.0 is limited to the designated CII systems as defined by CSA and CIIOs, not extending to other parts of the enterprise network unless explicitly included in the CII designation.</i> <i>Sources: Security By Design Annex C; Response-to-Feedback 2.2; clause 10.2.1; Response-to-Feedback 11.25</i>
Citation Accuracy	No	Largely (one document mislabel)
LLM Judge Score	0.11	0.56

Where B05 showed a right answer resting on wrong citations, B01 shows the more dangerous failure: a *confidently wrong verdict*. Hybrid retrieved only governance and network-control clauses, none about scope, so with nothing to anchor the boundary question it fell back on a plausible but incorrect “interconnectedness” argument, concluded that compliance extends to the whole network, and even invoked a Cybersecurity Act section 11(7) waiver citation that is neither relevant nor present in its context. That answer scored zero on verdict, justification, grounding, and citation, and failed the case at 0.11. GraphRAG’s concept mapping instead surfaced the digital-boundary scope cluster, with the decisive Response-to-Feedback paragraph 2.2, so the model reasoned correctly that compliance follows the CII’s digital boundary rather than the shared network and reached the correct “not applicable” verdict, passing at 0.56. As with B05, this is a single case on a benchmark the graph was designed around, GraphRAG’s own citations were imperfect (one clause mislabelled to the wrong document, hence citation correctness of only 1 of 3), and neither mode offered an actionable way forward; the full ablation of Section 17.4 remains the real test.

17.3 Observations & Plan

The primary hypotheses are that GraphRAG lifts **citation correctness** and the **cross-clause and multi-hop cases**, the two residuals that hybrid RAG left open, above the naïve-RAG baseline, without regressing the dimensions hybrid RAG already handles well (reasoning quality and scope appropriateness). The headline read-out will be the per-dimension and per-benchmark improvement of GraphRAG over hybrid RAG, presented in the same form as the Term-2 Native-versus-RAG comparison (Section 9) so that the two studies are directly comparable.

18 Research Arc (Prior Experiments)

The current GraphRAG build is the fourth iteration of the Term-3 retrieval work. Each earlier attempt was a genuine experiment that produced a decisive negative finding, and it is those findings, rather than a pre-committed plan, that shaped the current design. This is the project’s research-first method in action.

Iteration	Approach	Findings
1. Basic GraphRAG baseline	An un-governed graph built by free (schema-free) extraction with a lightweight model, retrieved through a graph database into the same model.	The extraction modelled the scenario, not the regulation (no clause-level concepts). On the single case tested it scored well below hybrid RAG, but the comparison was confounded because the baseline used very coarse chunks and no re-ranking. This motivated a governed, ontology-grounded graph.

2. Ontology-grounded graph	A curated CCoP ontology was locked and used to govern extraction, with clause nodes seeded and validated.	The ontology governed <i>extraction</i> but was never actually <i>queried</i> at retrieval time; retrieval remained ordinary chunk search with a thin graph decoration. A literature review then showed that retrieval-only graph use is not enough.
3. GraphCompliance	A reasoning-first redesign modelling each obligation as a structured unit, with a per-query context graph and a decision gate. Around 800 obligation units were built.	This design never beat hybrid RAG on the anchor case, and diagnosis showed the failure was a <i>ranking</i> problem, not the architecture: the decisive clause was present in the candidate pool but out-ranked by generic material. The obligation-unit model was the wrong investment for a ranking problem.
4. OMD-GraphRAG (current)	A shift from obligation reasoning to an ontology-guided concept graph with multi-channel retrieval (this section). Clauses link to the concepts they invoke, and concepts link to one another, so clauses bridge through shared concept hubs.	Cross-document bridging, the residual from Section 11, becomes native (the B05 password bridge; the B01 CII hub spanning all seven documents). The earlier obligation-unit graph was archived and removed. This is the current work.

OMD-GraphRAG and GraphCompliance are two different answers to the same diagnosis. GraphCompliance structures the regulation as obligations and reasons over them; OMD-GraphRAG structures it as an ontology-guided concept graph and fuses several retrieval channels. Term 3 tried the reasoning-first answer first, found that it stalled on ranking, and pivoted to the retrieval-fusion answer, carrying forward the earlier lessons (anchoring to the scenario, and carrying verbatim clause text) while dropping the obligation-unit model.

19 Next Steps

The focus for the remainder of Term 3 will be to further tune the retrieval methodology and resolve residual ranking issues. We will broaden the end-to-end checks beyond the two anchor cases to a wider slice, confirming that the ground-truth clauses appear in the top results across benchmarks. We have established a stratified subset of test cases that represent each of the 18 benchmarks under evaluation. We will run the ablation- GraphRAG vs Hybrid RAG vs LLM-only under the LLM judge, isolating citation-correctness and cross-clause-bridging improvements. In the process, we will also identify the gaps- benchmarks where the model scored relatively lower due to gaps in reasoning, hallucinations or insufficient grounding in CCoP facts. These may form the premise to perform another ablation study to provide the final grounding and domain-adaptation of the Primus model on CCoP 2.0.

-----END-----

20 References

- [1] J. Wang, H. Huang, X. Ge, J. Su, W. Liu, and S. Lian, "OMD-GraphRAG: Enhancing GraphRAG with ontology-guided extraction, multi-dimensional clustering and dual-channel fusion," arXiv preprint arXiv:2603.25152, 2026. [Online]. Available: <https://arxiv.org/abs/2603.25152>
- [2] J. Chung et al., "GraphCompliance: Aligning policy and context graphs for LLM-based regulatory compliance," arXiv preprint arXiv:2510.26309, 2025. [Online]. Available: <https://arxiv.org/abs/2510.26309>
- [3] H. de Martim, "An ontology-driven graph RAG for legal norms: A structural, temporal, and deterministic approach," arXiv preprint arXiv:2505.00039, 2025. [Online]. Available: <https://arxiv.org/abs/2505.00039>

21 Appendix

21.1 Retrieval Augmentation Techniques

The two LLM-based augmentation techniques referenced in Section 4.1 Contextual RAG (indexing time) and Section 5.1 HyDE Rewrite (query time) are documented in detail below.

21.1.1 Contextual RAG

What it is. Contextual Retrieval (Anthropic, 2024) is a chunk-augmentation technique that prepends each chunk with explanatory context generated by an LLM at indexing time. The augmented chunk replaces the original for embedding and reranking; the original text is preserved in metadata for downstream use.

Motivation. Standard dense retrieval embeds chunk text in isolation. A clause like *"5.2.2 The CHIO shall perform a review of all accounts..."* loses signal when stripped from its document and section context — the embedding becomes ambiguous between unrelated documents that use similar wording. Augmenting the chunk with a structural breadcrumb (*"This is from CCoP 2.0, Chapter 5: Identity and Access Management"*) and an LLM-generated summary block grounds the embedding in its parent context, materially improving cross-encoder discrimination on short user queries.

Implementation. Each chunk is augmented with two elements: (a) a structural breadcrumb (document → section → clause path) emitted by the chunker, and (b) an LLM-generated 2–3 sentence context block describing what the clause covers and how users might search for it. The contextualization model runs once at indexing time. The augmented form is what gets embedded; the production index is the contextualized v3 collection (ccop_clauses_contextual_v3).

Worked example — clause CCoP 2.0::5.3.1 (Privileged Account Management).

Original chunk text (792 chars), as it appears in the source PDF:

5.3.1 With respect to privileged accounts, the CHIO shall:

- (a) *Ensure that privileged access (i.e., administrative access) is granted only to selected accounts authorized to have such access;*
- (b) *Maintain an updated inventory of privileged accounts including details of the permissions and privileges assigned to each account;*

- (c) Implement multi-factor authentication where privileged accounts are used to access the CII, and where privileges are to be escalated to the level of privileged access (e.g., where the user seeks to obtain additional permissions on a system or network after an initial log-in); and
- (d) Ensure that privileged access is initiated from a cybersecurity hardened environment and transfer of data takes place over authorized connections.

Augmented form (1236 chars), what actually gets embedded in the production index:

[Doc: CCoP 2.0 | Ch: 5 | Sec: 5.3]

[Context: This section of CCoP 2.0 outlines the responsibilities of the CIIO regarding the management and security of privileged accounts, emphasizing the need for controlled access, inventory maintenance, multi-factor authentication, and secure environments. Users might search for terms like "privileged account management," "access control," "authentication requirements," or "cybersecurity protocols."]

5.3.1 With respect to privileged accounts, the CIIO shall:

[... original text appended verbatim ...]

The breadcrumb places the chunk in its document hierarchy; the context block introduces vocabulary a user query is more likely to use ("*privileged account management*", "*authentication requirements*") than the regulatory text itself. Retrieval is now against the union — clause text plus its lexical paraphrase — without altering the source-of-truth content.

21.1.2 HyDE Query Rewriting

What it is. HyDE (Hypothetical Document Embedding, Gao et al. 2022) is a query-side technique that uses an LLM to rewrite the user's natural-language query into a hypothetical answer document, which is then embedded for retrieval instead of the raw query.

Motivation. A bi-encoder retriever cannot expand acronyms via attention — "*MFA*" in a user query will not match "*multi-factor authentication*" in a clause body, even though the two refer to the same concept. The same gap applies to colloquial-vs-formal phrasing ("*Can my system get an exception?*" vs. "*The CIIO may apply for a waiver under Section 11(7)*"). HyDE bridges this gap by having an LLM generate a hypothetical clause-style answer that contains the formal vocabulary expected in the corpus, so the embedding-space comparison is between regulatory-style text on both sides.

Implementation. The original query is sent to gpt-4o-mini via OpenRouter with a prompt instructing it to produce 2–3 sentences in formal regulatory style using CCoP 2.0 vocabulary ("*the CIIO shall...*", "*the Commissioner may...*", "*waiver*", "*compliance*") without preamble. The hypothetical clause becomes the embedding query. The original user query is preserved in graph state — it is needed downstream for the generator's context (the user wants their question answered, not the hypothetical).

Worked example — test case B22-001 (Waiver / Exception Reasoning).

User query (informal, with acronym):

Can a CII operator apply for a waiver for MFA requirements on legacy SCADA systems?

HyDE rewrite produced by gpt-4o-mini, embedded for retrieval in place of the user query:

The CIIO shall submit a formal request for a waiver from the multi-factor authentication requirement under section 11(7) of the Act, with justifications supporting the request for legacy operational technology systems where compliance is technically infeasible.

The actual answer clause is CCoP 2.0::1.6.2:

"A CIIO can request for waiver from specific provisions of this Code under section 11(7) of the Act by submitting a written request to the Commissioner with the justifications supporting the request."

The user's query says "*MFA*" and "*apply for a waiver*"; the corpus says "*multi-factor authentication*" and "*request for waiver*". A bi-encoder cannot bridge "*MFA*" → "*multi-factor authentication*" via attention alone. The HyDE rewrite replaces "*MFA*" with the expanded form and switches "*Can a CII operator apply*" → "*The CIIO shall submit a formal request*" — token-level overlap with Clause 1.6.2 is now substantial. In the production retrieval trace for B22-001, Clause 1.6.2 surfaces at **dense rank 1, similarity 0.738** with HyDE; without it, the same query would not lexically match.

21.2 LLM as Judge Reference

21.2.1 Anchor Scale, Ratios, and Composite Score

The 0–3 anchored scale used by every dimension:

Score	Level	Definition
0	Incorrect	Contains factually wrong regulatory information, fabricated claims, or fundamentally misinterprets the requirement
1	Partial	Correct core but incomplete — missing key regulatory details, clauses, or context
2	Complete	Fully consistent with the expected answer; all required regulatory points covered accurately
3	Exceeds	Fully consistent and provides additional correct, relevant information

D3 and D6 use count-based ratios over atomic units (claims for D3, citations for D6) rather than holistic judgement:

D3 ratio = SUPPORTED / (SUPPORTED + UNSUPPORTED + CONTRADICTED)

D6 ratio = (CORRECT + 0.5 × IMPRECISE) / (CORRECT + IMPRECISE + MISATTRIBUTED + FABRICATED)

ratio = 1.0 → score 3

ratio ≥ 0.67 → score 2

ratio ≥ 0.34 → score 1

ratio < 0.34 → score 0

0 claims/citations → score 1 (neutral)

The composite score is the equally weighted normalized average:

composite = $\Sigma(\text{score}_i \times \text{weight}_i) / (3.0 \times \Sigma \text{weight}_i)$

With six dimensions at weight 0.5 each, the composite reduces to the simple mean of the six dimension scores divided by 3, ranging 0.0–1.0.

21.2.2 Judge Prompt and Output Schema

The judge system prompt establishes it as a compliance auditor that must score strictly from the supplied ground truth. The load-bearing instruction is: *"If a claim in the response is not verifiable against the provided ground truth AND cannot be traced to a clause whose actual text is shown below, treat it as ungrounded. Leniency not based on ground truth is a scoring error."* Without this, judge models drift toward giving the benefit of the doubt to plausible-looking regulatory content, inflating scores on responses that confidently cite plausible but unverifiable material.

The prompt assembles seven ground-truth fields per test case, each serving a distinct dimension:

Field	Source	Used for
question	Test case input	D4 constraint extraction, D2 reasoning chain check
expected_response	Ground truth	D1 verdict comparison
key_facts	Ground truth (CRITICAL/IMPORTANT tiers)	D1, D2 — missing CRITICAL facts must score lower than missing IMPORTANT ones
clause_reference	Ground truth	D6 — canonical clauses the expected answer cites
expected_citations_text	Programmatically resolved from clause inventory	D3 — actual text of expected clauses for substantive grounding
citation_verifications	Programmatically classified from response (Appendix C.3)	D3, D6 — per-citation status
forbidden_claims + hallucination_patterns	Ground truth	D3 — automatic CONTRADICTED classification on match

The judge returns a single JSON object:

```
{
  "dimensions": [
    {"dimension": "verdict_accuracy",      "score": 0-3, "weight": 0.5},
    {"dimension": "justification_quality",  "score": 0-3, "weight": 0.5},
    {"dimension": "factual_grounding",     "score": 0-3, "weight": 0.5},
    {"dimension": "scope_appropriateness",  "score": 0-3, "weight": 0.5},
    {"dimension": "actionable_way_forward", "score": 0-3, "weight": 0.5},
    {"dimension": "citation_correctness",   "score": 0-3, "weight": 0.5}
  ],
  "justification": "1-2 sentences per dimension, with required formats per dimension",
  "confidence": 0.0-1.0
}
```

Justification format is constrained per dimension to keep explanations auditable: D3 requires count-only statements ("4 SUPPORTED, 1 UNSUPPORTED, 0 CONTRADICTED → ratio 0.8 → D3=2") plus one example per CONTRADICTED claim; D4 requires that any constraint violation be named explicitly; D5 must declare whether the failure mode is *(a) no steps* or *(b) infeasible steps*; D6 uses the same count-and-ratio form as D3.

21.2.3 Citation Verification and Classification Taxonomy

D3 (claim-level) and D6 (citation-level) depend on programmatic verification of the response's citations against the corpus, not on the judge's recognition of clause IDs. The verification infrastructure is initialised at judge startup with three layers:

(a) Doc-keyed clause inventory. A canonical inventory (`src/rag/ingestion/fixtures/clause_inventory.json`) enumerates every clause ID across the seven source documents, stored doc-keyed (`source_doc → set[clause_id]`) so a clause format valid for one document is not spuriously credited or blamed on another.

(b) Document alias normalization. Models write document names as natural variants — *"the Act"*, *"CCoP"*, *"Audit Guidelines"* — which are mapped to canonical inventory keys via an alias table. Citations whose document does not resolve are classified EXTERNAL (cross-document references like NIST CSF or ISO 27001) and excluded from D6 rather than penalized as fabricated.

(c) Clause-text cache. Actual clause text is pre-loaded from the Qdrant vector store at startup, keyed by (`canonical_doc`, `clause_id`). This is what enables MISATTRIBUTED detection: a citation pointing to a real clause that doesn't actually contain the claimed content fails D6 even though its ID exists. When Qdrant is unreachable, the judge degrades gracefully to inventory-only checks (existence works; misattribution disabled).

Each citation is classified as exactly one of:

EXTERNAL → outside 7-doc corpus → excluded from D6

FABRICATED → claims a corpus clause that doesn't exist → counted, scored 0

EXISTS:

CORRECT → ID exists, description matches clause text

IMPRECISE → ID exists, description partially matches (counted at 0.5 weight)

MISATTRIBUTED → ID exists but description does not match

Sub-letter precision is enforced: 5.3.1(c) is FABRICATED when only 5.3.1(a) and 5.3.1(b) exist in the corpus, even though the parent clause 5.3.1 is real. This catches a class of hallucination where models invent plausible sub-clauses by extrapolating numbering patterns.

21.2.4 Runtime Configuration and Error Handling

The judge runs against an external model via OpenRouter (OpenAI-compatible chat completions API). Using a different model family from the system under test avoids self-evaluation bias on responses generated by the local Llama-Primus-Reasoning model.

Setting	Default	Configurable via
Primary judge model	qwen/qwen3-235b-a22b-07-25	CCOP_JUDGE_PRIMARY_MODEL
Secondary judge model	openai/gpt-4o-mini	CCOP_JUDGE_SECONDARY_MODEL
Sampling temperature	0.2	CCOP_JUDGE_TEMPERATURE
JSON retry attempts	3	CCOP_JUDGE_JSON_RETRY_ATTEMPTS
Per-call timeout	(settings)	CCOP_JUDGE_TIMEOUT
OpenRouter API retries	(settings)	CCOP_JUDGE_MAX_RETRIES

Temperature is pinned at 0.2 — low enough to suppress sampling variance, high enough to allow chain-of-thought without collapsing to deterministic output. Sampling at temperature 0 was avoided because some judge models exhibit degenerate outputs (repetition loops, premature termination) at the extreme.

Two safeguards apply at runtime:

5. **JSON parse retries.** When the judge returns malformed JSON (a recurring failure mode of long-context generation), the service re-issues the call up to `CCOP_JUDGE_JSON_RETRY_ATTEMPTS` times (default 3, comprising the initial call plus two retries). Each retry is logged through the structured logger so retry rates are observable per run rather than silently absorbed.
6. **Skip-and-flag, not fallback.** When all retries are exhausted, the test case receives a `JudgeEvaluation` with `judge_error=True` and zero scores, and is excluded from aggregate metrics rather than scored as a failure. This prevents a malformed-JSON bug — judge infrastructure issue, not model performance issue — from contaminating the model's measured score. Run summaries report the skipped count separately.

The framework supports a two-judge methodology in which the secondary judge is invoked alongside the primary on designated *measurement snapshots* — runs explicitly marked for human-review comparison. Per-test-case evaluation runs use the primary judge only to keep cost bounded.

21.3 Iteration History

This appendix records the qualitative iteration history behind Section 6.1 (Retrieval Pipeline) and Section 7.2(LLM as Judge). Headlines are summarized in Section 8.1 and Section 8.2, the full row-level history is below.

21.3.1 Retrieval Experiments and Ablations

The production retrieval stack documented in Section 6.1 is the result of 41 lab experiments, of which the headline ones are summarised below. Many of the 41 were small-grained sweeps (top_k variations, embedding model substitutions, RRF weight tuning) that are not narrated individually; the table captures the experiments that changed the architecture or settled a methodological question.

#	Iteration	Observation	Outcome	Motivated next
1	Baseline: dense bi-encoder retrieval, single-pass top-k, no reranker	ctx_recall plateaued at low values on multi-hop questions; correct clauses present in top-50 but not top-5	Insufficient as final stack	Add cross-encoder reranking
2	Cross-encoder reranker added (ms-marco-MiniLM-L12-v2)	Reranker frequently demoted correct clauses on short queries (e.g. B03 5.3.1 went from dense rank 7 → CE rank 12) — the bi-encoder's weighted token overlap was being overridden by the cross-encoder's exact-token bias	Kept reranker; switched model	Larger, regulatory-aware cross-encoder
3	Cross-encoder upgraded to bge-reranker-large	Discrimination improved on long queries, still degraded on short acronym-heavy queries	Kept	Address acronym expansion problem upstream
4	Exp #14 — chunk contextualization (breadcrumb + LLM-generated context appended at indexing)	Augmented chunks gave the cross-encoder regulatory anchors to lock onto; CE discrimination on short queries improved materially	Adopted	Iterate contextualization quality
5	Exp #15 — pass original_text to CE pairs	Reverted; v3 contextualization (clean acronym expansion only) produced cleaner augmented text that improved CE discrimination over original_text	Discarded	Use augmented_text for CE pairs
6	Exp #17 — HyDE query rewriting via gpt-4o-mini	Bridged acronym expansion ("MFA" → "multi-factor authentication") which neither bi-encoder nor cross-encoder could perform via attention alone	Adopted	Fold into Exp #41 production stack

#	Iteration	Observation	Outcome	Motivated next
7	Exp #16 / #33 — parent-child auto-merge	Sibling sub-clauses (5.3.1(a), (b), (c)) frequently appeared in the top-window separately; merging their content under one anchor improved coverage on multi-clause answers without bloating prompt token count beyond the merge cap	Adopted	Add score-ratio gate to prevent weak siblings being bundled in
8	Exp #28 — RRF ensemble of dense rank + CE rank	Pure CE ranking lost the bi-encoder's recall on cases where the cross-encoder under-weighted regulatory tokens; RRF (K=60, w_dense=1.0, w_ce=1.5) preserved both signals	Adopted	Iterate weights via sweep
9	TOC filter (dot-leader heuristic, 2026-04-27)	Identified 7 TOC/index chunks (preambles of 5 docs + a misclassified Risk Assessment Guide page) that polluted top-K with 6–15K chars of noise per query	Adopted	One-shot fix; not parameterized further
10	top_n reduction 8 → 3 (2026-04-27)	At top_n=8, prompts hit 44K chars and reranker scores were clustered at 0.000–0.080 with no clear winner on short queries; verbosity grew without precision gain	Adopted	Investigate retrieval recall@C metric (top_n at corpus cardinality) for honest recall reporting
11	Exp #41 — full migration to production	All adopted experiments combined (HyDE → dense on contextual_v3 → bge-reranker on original_text → RRF → parent-child merge → top_n=8) produced R@C 0.510 / R@8 0.600 on the 30-case lab subset, vs. baseline lab benchmark within ~5pts	Migrated to mainline (commit 92463c1)	Tune top_n in production (Exp #11→#10 above)
12	Cybersecurity Act citation-format mismatch	Statute uses Section 11(7) while CCoP uses 5.3.1(c); the same regex/inventory logic mishandled cross-document citations	Patched in inventory + alias normalization	Audit citation classification surface end-to-end
13	RAG generation prompt — "Retrieved Context" parroting	Model began every response with "Based on the retrieved context..." — the system prompt's framing was being repeated verbatim	Strip seed phrases from system prompt rather than add forbidden-phrase rules	Generalize: address cause, not symptom
14	Sources footer convention (<doc>: <clause> per line)	Free-form citation styles were unparseable by the resolver; structured footer enabled deterministic post-generation citation extraction	Adopted	Couple footer to D6 verification chain

#	Iteration	Observation	Outcome	Motivated next
15	Removed "Prioritize answers grounded in passages" over-instruction	Caused the model to reject perfectly valid inferential answers when the exact phrasing wasn't in the passages	Discarded	Trust the rubric (D3 claim-level grounding) instead of over-constraining the generator

21.3.2 Judge Experiments and Ablations

The LLM Judge evolved through twelve iterations between the original 5-dimension cliff design and the production 6-dimension equal-weight rubric documented in Section 7.2.

#	Iteration	Observation	Outcome	Motivated next
1	Original 5-dim rubric with cliff weights (D1=0.4, D2=0.25, D3=0.2, D4=0.1, D5=0.05)	A small change in D1 score moved the composite by 0.13 points, while D5 was effectively unscored — the weight schedule made D1 deterministic of the outcome	Discard cliff weighting	Equal weights across dimensions
2	Equal weight 0.2 across 5 dimensions	Composite became responsive to all dimensions; revealed that D3 was conflating two distinct signals (claim-level grounding vs citation-level correctness)	Kept equal weighting	Split D3 into two dimensions
3	Split D3 → D3 (factual_grounding, claim-level) + D6 (citation_correctness, citation-level)	A response with perfect IDs but wrong descriptions, and a response with weak IDs but correct substantive claims, both became scoreable distinctly	Adopted; rubric grew to 6 dimensions	Rebalance weights to sum
4	Six dimensions × equal weight 0.5 (composite normalized to 0–1)	Stable across the test corpus; composite range reflected actual response variation	Adopted as production rubric	Address judge-side reliability issues
5	JSON parse failures on long-context judge calls	Roughly 3–5% of calls produced malformed JSON; treating these as score-zero contaminated the model's measured score with judge infrastructure noise	Add JSON parse retries	Make retry rates observable
6	JSON retry implemented (default 3 attempts)	Most parse failures resolved within retry budget; residual failures concentrated on a small set of test cases with very long expected responses	Adopted	Surface retry rates for audit
7	Structured logging for retry events	Retry rate per run became visible in logs alongside test-case	Adopted	Decide how to handle

#	Iteration	Observation	Outcome	Motivated next
		progress; previously-silent failures became attributable to specific (judge_model, test_case) pairs		terminal retry-exhaustion cases
8	Skip-and-flag on terminal retry failure	Test case marked judge_error=True and excluded from aggregate metrics rather than scored zero; run summary reports skipped count separately	Adopted	Audit skipped cases per run for systemic patterns
9	Claim-level grounding refactor — D3 procedure changed from holistic 0–3 judgement to count-based ratio over the 5 most load-bearing claims	Inter-judge variance on D3 dropped substantially; per-judge "leniency" became less of a factor	Adopted	Apply count-based pattern to D6
10	Count-based ratio for D6 (CORRECT, IMPRECISE, MISATTRIBUTED, FABRICATED)	D6 became as reliable as D3 by the same mechanism — judge compares against deterministic verification output rather than holistic impression	Adopted	Compress D3 procedure to keep prompt tractable
11	Simplified D3 procedure (compressed from ~30 lines to ~15)	Long procedural blocks in the rubric had been confusing the judge — it occasionally followed a partial procedure and produced inconsistent counts. Compression to the essential steps recovered consistency	Adopted	Apply same compression to D4/D5
12	Anti-lenieny instruction in system prompt — "Leniency not based on ground truth is a scoring error"	Without this instruction, judge models drifted toward giving the benefit of the doubt to plausible-looking regulatory content; with it, scores on confidently-cited but unverifiable material dropped to expected ranges	Adopted	Final 6-dim equal-weight production rubric

21.4 Runtime Evaluation- Framework in Action



Figure 4: CCoP Evaluation framework in action, showing model response and evaluation for a RAG augmented pipeline, focusing on assessment of model grounding to CCoP facts